

Yes. This is the **DDPM (Denoising Diffusion Probabilistic Models)** formulation. The key is to understand that the paper defines **two opposite Markov chains**:

1. **Forward process**: start with a real image (x_0), gradually add Gaussian noise until it becomes almost pure noise (x_T).
2. **Reverse process**: start with pure Gaussian noise (x_T), and learn to gradually remove the noise until we obtain a generated sample (x_0).

I'll unpack the equations from the ground up.

1. The central idea

Suppose (x_0) is a real image.

For example,

```
[  
x_0 =  
\text{a photograph of a cat}.  
]
```

The diffusion model creates a sequence

```
[  
x_0, x_1, x_2, \ldots, x_T.  
]
```

The sequence looks like:

```
[  
\boxed{  
\text{clean image}  
\rightarrow  
\text{slightly noisy}  
\rightarrow  
\text{more noisy}  
\rightarrow  
\cdots  
\rightarrow  
\text{pure noise}  
}]
```

```
}  
]
```

That is the **forward diffusion process**.

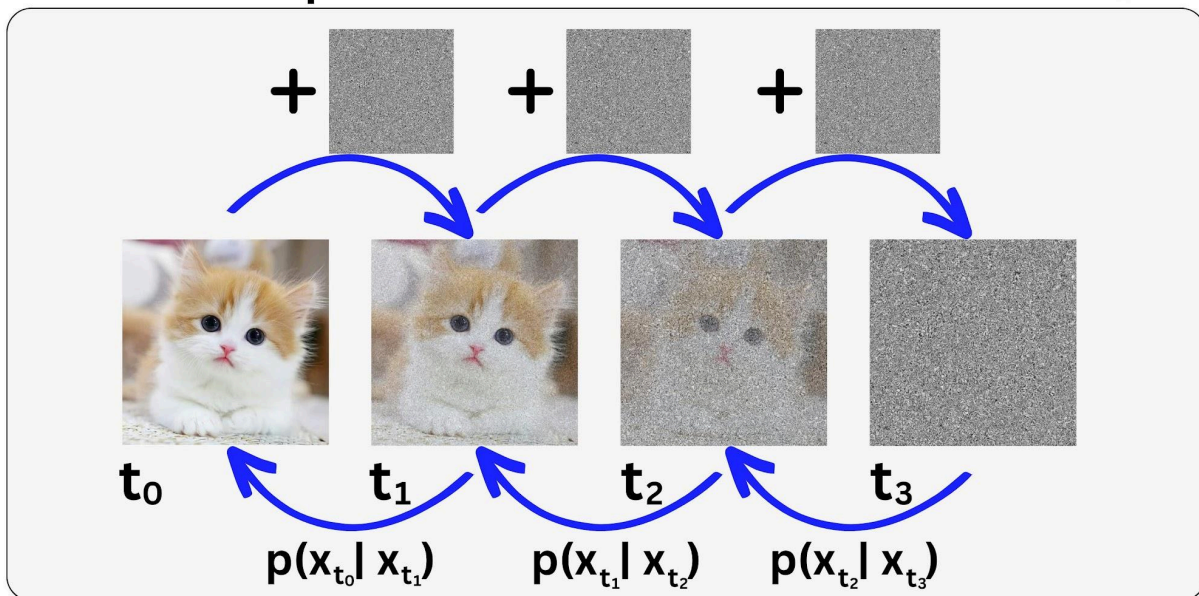
Then the model learns the reverse:

```
[  
  \boxed{  
    \text{pure noise}  
    \rightarrow  
    \text{slightly less noisy}  
    \rightarrow  
    \cdots  
    \rightarrow  
    \text{clean image}  
  }  
]
```

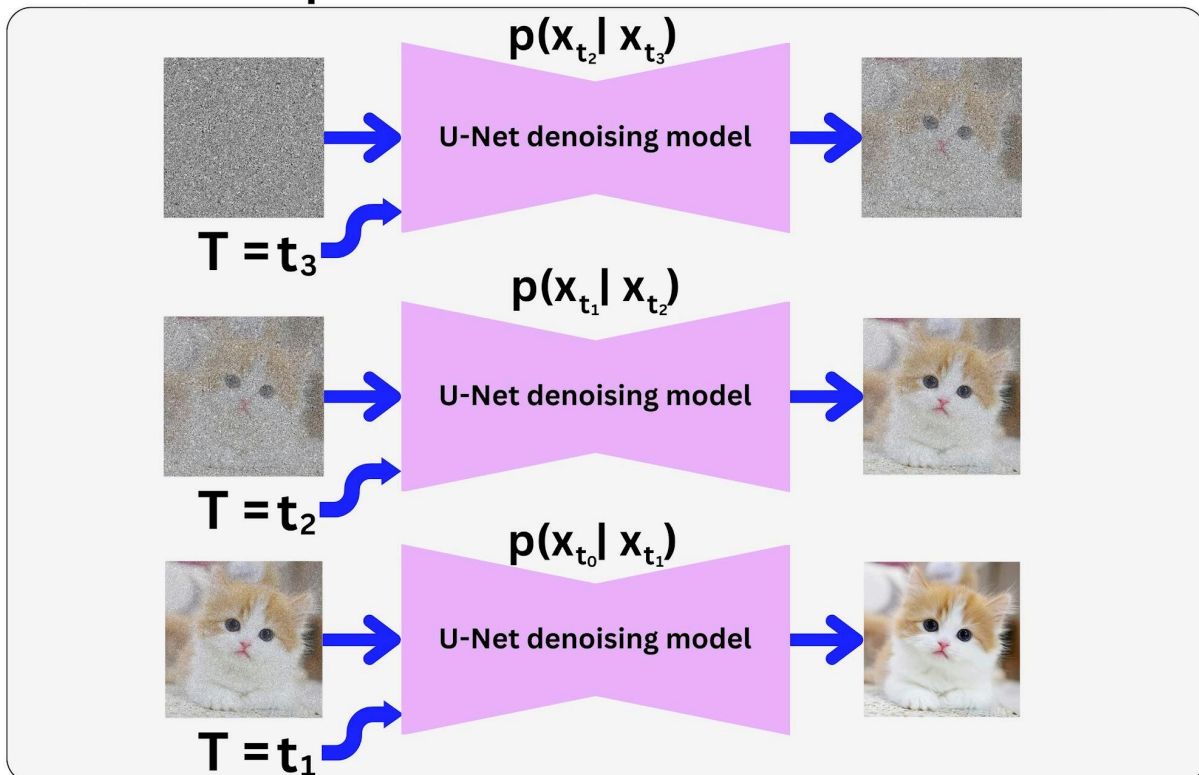
That is the **reverse generative process**.

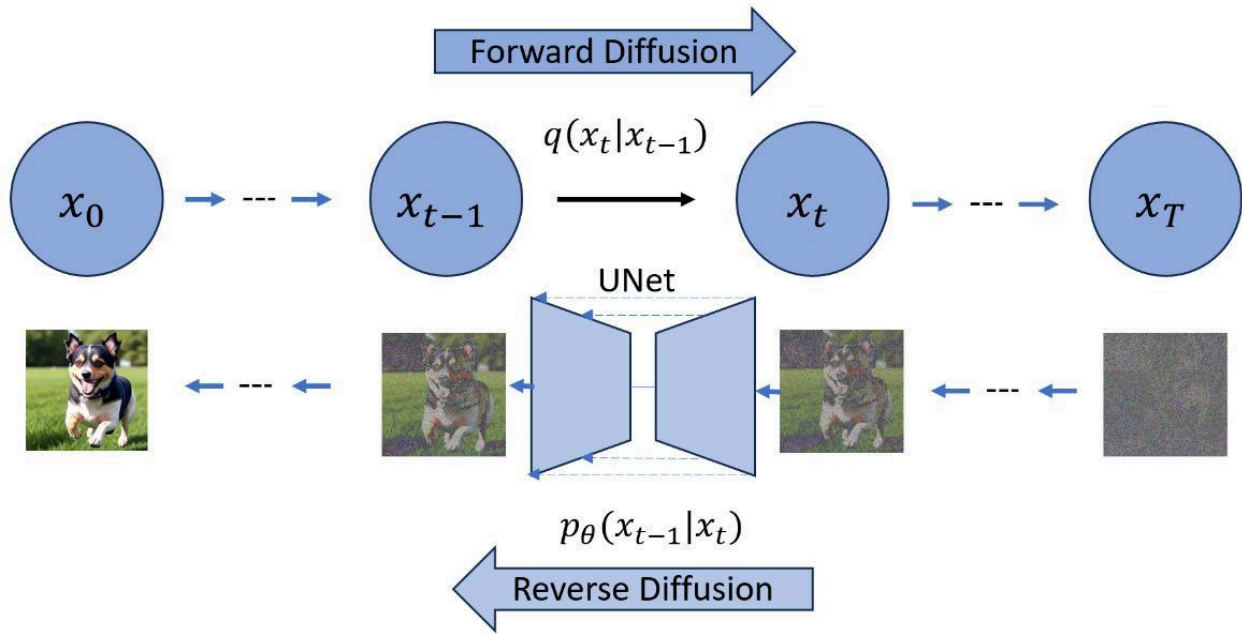
The Forward process

TheAiEdge.io



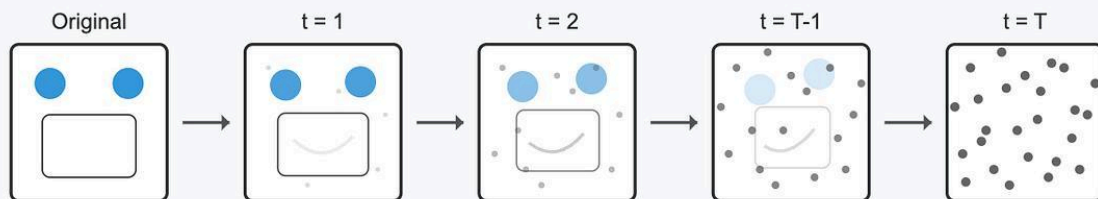
The Reverse process



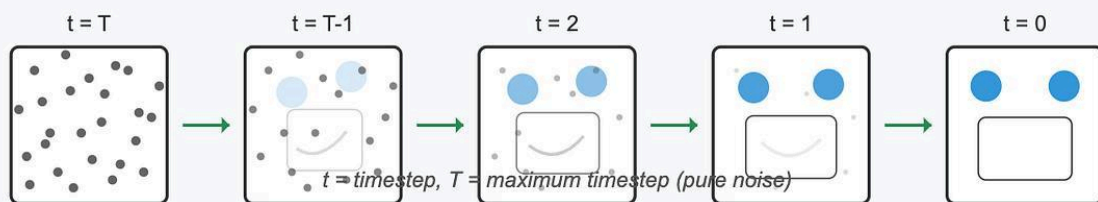


Diffusion Models: Forward and Reverse Processes

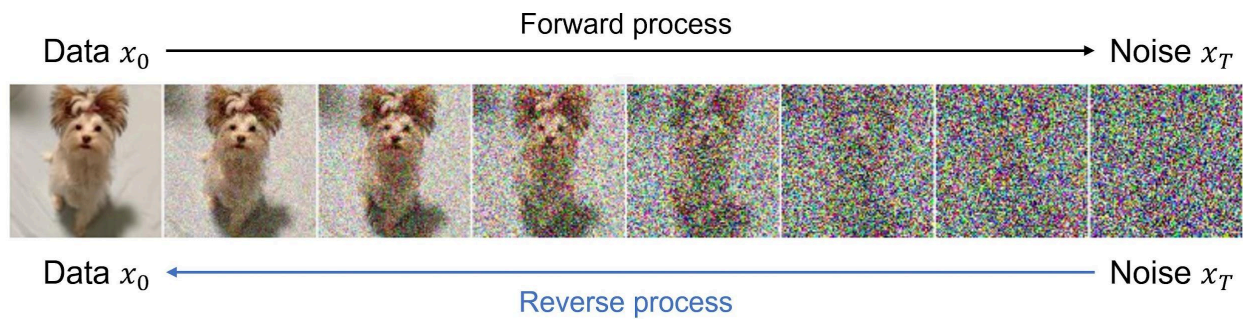
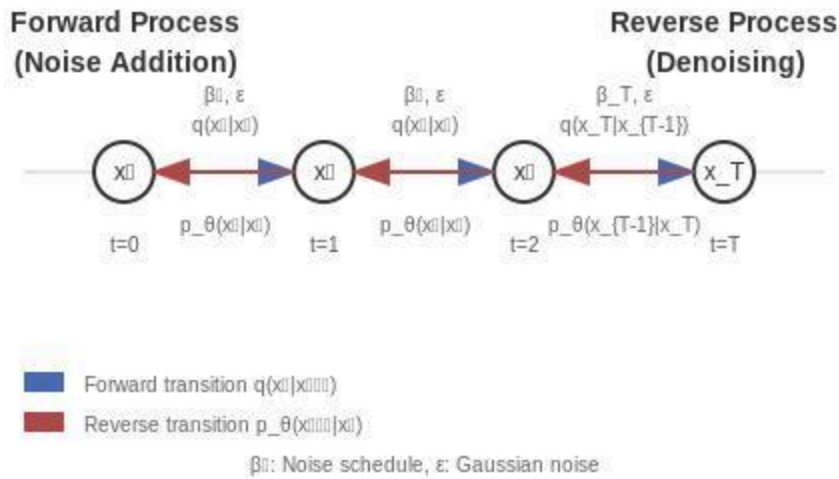
Forward Process (Training): Gradually Adding Noise



Reverse Process (Generation): Gradually Removing Noise



DDPM Forward and Reverse Processes



The remarkable part is that the forward process is **not learned**. We define it ourselves.

The reverse process **is learned**.

2. Why is it called a probabilistic model?

Because every transition is probabilistic.

Instead of saying

$$\begin{aligned} &[\\ &x_t = f(x_{t-1}), \\ &] \end{aligned}$$

we say

$$\begin{aligned} &[\\ &x_t \sim q(x_t | x_{t-1}). \\ &] \end{aligned}$$

So the next state is sampled from a probability distribution.

The forward process is a Markov chain:

$$\begin{aligned} &[\\ &x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \cdots \rightarrow x_T. \\ &] \end{aligned}$$

"Markov" means that the next state depends only on the immediately previous state:

$$\begin{aligned} &[\\ &q(x_t | x_{0:t-1}) = q(x_t | x_{t-1}). \\ &] \end{aligned}$$

3. What are $(x_0, x_1, \dots, x_T)$?

This is extremely important.

(x_0)

The original data.

For an image:

[
x_0 \in \mathbb{R}^{H \times W \times C}.
]

For example,

[
x_0 = \text{clean cat image}.
]

(x_1)

A slightly noisy version.

(x_2)

A little more noisy.

(x_T)

Almost pure Gaussian noise.

So:

[
x_0 = \text{data}
]

and

[
x_T \approx \mathcal{N}(0, I).
]

The intermediate (x_t)'s are called **latent variables** because they are not the final observed data.

4. Why does the paper call this a latent-variable model?

The paper says

$$[\begin{aligned} & p_{\theta}(x_0) \\ & := \\ & \int p_{\theta}(x_{0:T}), dx_{1:T}. \end{aligned}]$$

What does this mean?

The model is interested in the probability of the observed data (x_0).

But there are hidden variables

$$[\begin{aligned} & x_1, x_2, \dots, x_T. \end{aligned}]$$

Therefore the model has a joint distribution

$$[\begin{aligned} & p_{\theta}(x_0, x_1, \dots, x_T). \end{aligned}]$$

To obtain the probability of (x_0), we integrate out all the latent variables:

[

$$p_{\theta}(x_0)$$

$$[\begin{aligned} & \int \\ & p_{\theta}(x_0, x_1, \dots, x_T) \\ & dx_1 \cdots dx_T. \end{aligned}]$$

This is exactly the same basic idea as other latent-variable models such as VAEs.

5. The reverse process

The paper defines

[

$$p_{\theta}(x_{0:T})$$

$$p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1}|x_t).$$

Let's understand every piece.

First:

$$p(x_T) = \mathcal{N}(x_T; 0, I).$$

This says:

At the end of the diffusion chain, we assume that (x_T) comes from a standard Gaussian.

So generation starts from

$$x_T \sim \mathcal{N}(0, I).$$

Then we repeatedly sample

$$x_{T-1} \sim p_{\theta}(x_{T-1}|x_T)$$

then

$$x_{T-2} \sim p_{\theta}(x_{T-2}|x_{T-1})$$

and eventually

$$[x_0 \sim p_{\theta}(x_0|x_1).]$$

Therefore generation is

$$[\boxed{x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_1 \rightarrow x_0}]$$

6. Why is it called the reverse process?

Because it reverses the direction of the forward diffusion.

Forward:

$$[x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_T.]$$

Reverse:

$$[x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_0.]$$

The forward process is known.

The reverse process is learned.

7. The Gaussian reverse transition

The paper defines

$$[$$

$p_{\theta}(x_{t-1}|x_t)$

```
\mathcal{N}
\left(
x_{t-1};
\mu_{\theta}(x_t,t),
\Sigma_{\theta}(x_t,t)
\right).
```

This says that given a noisy image (x_t), the model predicts a Gaussian distribution for the slightly less noisy image (x_{t-1}).

The neural network determines

```
[
\mu_{\theta}(x_t,t)
]
```

and potentially

```
[
\Sigma_{\theta}(x_t,t).
]
```

So conceptually:

```
[
x_t
\overset{\text{neural network}}{\longrightarrow}
\mu_{\theta}, \Sigma_{\theta}
]
```

and then

```
[
x_{t-1} \sim
\mathcal{N}(\mu_{\theta}, \Sigma_{\theta}).
]
```

8. Why does the model need (t)?

Because the amount of noise depends on the timestep.

For example:

At (t=10)

The image might contain lots of recognizable structure.

At (t=500)

It may be heavily corrupted.

At (t=1000)

It may be almost pure noise.

Therefore the network must know:

"How noisy is this input?"

That's why the network receives both

[
x_t
]

and

[
t.
]

Hence:

[
 $\mu_{\theta}(x_t, t)$.
]

Modern diffusion models typically encode (t) using a **time embedding**.

9. The forward process

Now we come to Equation (2):

[

$$q(\mathbf{x}_{1:T}|\mathbf{x}_0)$$

$$\prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1}).$$

This is the forward diffusion process.

Each transition is

[

$$q(\mathbf{x}_t|\mathbf{x}_{t-1})$$

$$\mathcal{N}\left(\mathbf{x}_t;\sqrt{1-\beta_t}\mathbf{x}_{t-1},\beta_t\mathbf{I}\right).$$

They define

$$\alpha_t=1-\beta_t.$$

Therefore:

[

$$q(\mathbf{x}_t|\mathbf{x}_{t-1})$$

```

\mathcal{N}
\left(
x_t;
\sqrt{\alpha_t}x_{t-1},
(1-\alpha_t)I
\right).
]

```

10. What exactly does β_t do?

β_t controls how much noise is added at timestep t .

Suppose

```

[
\beta_t=0.01.
]

```

Then approximately 1% of the variance is introduced as noise at that step.

The mean becomes

```

[
\sqrt{1-\beta_t}x_{t-1}.
]

```

The covariance becomes

```

[
\beta_t I.
]

```

So we can write the transition using the reparameterization form:

```

[
\boxed{
x_t=
\sqrt{1-\beta_t}x_{t-1}
+
\sqrt{\beta_t}\epsilon
}
]

```

where

```
[  
\epsilon\sim\mathcal{N}(0,I).  
]
```

Or using (α_t) :

```
[  
\boxed{  
x_t=  
\sqrt{\alpha_t}x_{t-1}  
+  
\sqrt{1-\alpha_t}\epsilon.  
}  
]
```

This equation is fundamental.

11. Intuition behind the forward equation

Suppose an image is represented by some vector (x_{t-1}) .

We multiply it by

```
[  
\sqrt{\alpha_t}  
]
```

to slightly reduce its signal.

Then we add

```
[  
\sqrt{1-\alpha_t}\epsilon.  
]
```

Therefore:

[

\text{new image}

\text{remaining signal}
+
\text{new noise}.
]

Repeated many times:

[
 $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_T$.
]

The original signal gets progressively destroyed.

12. Why is the forward process fixed?

This is one of the major differences from the reverse process.

We choose the schedule

[
 $\beta_1, \beta_2, \dots, \beta_T$.
]

For example, we might define

[
 $\beta_1 < \beta_2 < \dots < \beta_T$.
]

Then the forward process is completely known.

There is no neural network involved.

This is extremely useful because we can generate noisy training examples ourselves.

13. The crucial closed-form equation

The paper defines

$$\alpha_t = 1 - \beta_t$$

and

$$\bar{\alpha}_t$$

$$\prod_{s=1}^t \alpha_s.$$

$$\prod_{s=1}^t \alpha_s.$$

Then it derives

$$\boxed{$$

$$\boxed{$$

$$q(\mathbf{x}_t | \mathbf{x}_0)$$

$$\mathcal{N} \left(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I} \right).$$

This equation is one of the most important equations in DDPM.

14. Why is Equation (4) so important?

Without it, to create (x_t) , we would have to do:

[
 $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_t$.
]

That's potentially expensive.

But Equation (4) says:

You can jump directly from (x_0) to any (x_t) .

Specifically:

[

$\boxed{}$

x_t

$$\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

]

where

[
 $\epsilon \sim \mathcal{N}(0, I)$.
]

Therefore, during training:

1. Take a clean image (x_0) .
2. Randomly select (t) .
3. Generate (x_t) directly.
4. Train the network to reverse the corruption.

No need to generate

[
 $x_1, x_2, \dots, x_{t-1}$.
]

This is why diffusion training can be efficient.

15. What does $\bar{\alpha}_t$ mean intuitively?

Remember:

[

$\bar{\alpha}_t$

$\alpha_1 \alpha_2 \dots \alpha_t$.
]

Since

[
 $\alpha_t = 1 - \beta_t$,
]

$\bar{\alpha}_t$ represents roughly how much of the original signal survives after all noise additions up to timestep (t).

Thus:

[
 $\sqrt{\bar{\alpha}_t} x_0$
]

is the remaining signal.

And

$$\left[\sqrt{1 - \bar{\alpha}_t} \epsilon \right]$$

is the accumulated noise.

So:

[

\boxed{

x_t

$$\underbrace{\sqrt{\bar{\alpha}_t} x_0}_{\text{signal}} + \underbrace{\sqrt{1 - \bar{\alpha}_t} \epsilon}_{\text{noise}}$$

This decomposition is central to understanding diffusion models.

16. What happens when (t) becomes large?

Initially:

$$\left[\bar{\alpha}_t \approx 1 \right]$$

Therefore:

$$\left[x_t \approx x_0 \right]$$

At a very large (t):

$$\begin{bmatrix} \bar{\alpha}_t \approx 0. \end{bmatrix}$$

Therefore:

$$\begin{bmatrix} x_t \approx \epsilon. \end{bmatrix}$$

So:

$$\begin{bmatrix} x_T \approx \mathcal{N}(0, I). \end{bmatrix}$$

This explains why the generative process can start from standard Gaussian noise.

17. Now comes the difficult part: training

The goal is to learn

$$\begin{bmatrix} p_{\theta}(x_0). \end{bmatrix}$$

Ideally, we'd maximize

$$\begin{bmatrix} \log p_{\theta}(x_0). \end{bmatrix}$$

Equivalently, minimize

$$\begin{bmatrix} -\log p_{\theta}(x_0). \end{bmatrix}$$

But directly calculating

[

$p_{\theta}(x_0)$

$\int p_{\theta}(x_{0:T}) dx_{1:T}$
]

is difficult because we would need to integrate over all the latent variables.

This is exactly where **variational inference** enters.

18. Connection to VAE

This part should look familiar if you've studied VAEs.

In a VAE, we have

[
 $p_{\theta}(x, z)$
]

and approximate posterior

[
 $q_{\phi}(z|x)$.
]

We use a variational lower bound:

[
 $\log p_{\theta}(x)$
 $\geq$
 $\mathbb{E}_q$
 $\left[\log \frac{p_{\theta}(x, z)}{q_{\phi}(z|x)} \right]$.
]

DDPM uses essentially the same variational-inference idea.

But instead of one latent variable (z), we have:

```
[
x_1,x_2,\ldots,x_T.
]
```

And importantly, the approximate posterior is **fixed**:

```
[
q(x_{1:T}|x_0).
]
```

There are no learnable (ϕ) parameters in this forward process.

19. Equation (3)

The paper writes:

```
[
\mathbb{E}_q[-\log p_{\theta}(x_0)]
\leq
\mathbb{E}_q
\left[
-\log
\frac{p_{\theta}(x_{0:T})}
{q(x_{1:T}|x_0)}
\right].
]
```

The right-hand side is the **variational upper bound on negative log likelihood**.

This is equivalent to saying:

```
[
-\log p_{\theta}(x_0)
\leq
L.
]
```

So (L) is a loss/bound that we can optimize.

20. Why does the ratio appear?

Consider:

$$\left[\frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} \right]$$

Taking logarithm:

$$\left[\right.$$
$$\log$$
$$\frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)}$$
$$\log p_{\theta}(x_{0:T})$$
$$\log q(x_{1:T}|x_0).$$
$$\left[\right.$$

Taking negative:

$$\left[\right.$$
$$-\log$$

$$\frac{p_{\theta}(x_{0:T})}{$$

$$q(x_{1:T}|x_0)}$$

$$-\log p_{\theta}(x_{0:T}) \\ + \\ \log q(x_{1:T}|x_0).$$

This is the standard variational-inference structure.

21. Substitute the Markov chains

We have:

[

$$p_{\theta}(x_{0:T})$$

$$p(x_T) \\ \prod_{t=1}^T \\ p_{\theta}(x_{t-1}|x_t)$$

and

[

$$q(x_{1:T}|x_0)$$

$$\prod_{t=1}^T q(x_t|x_{t-1}).$$

Therefore:

[

L

$\mathbb{E}_q$

$\left[$

$-\log p(x_T)$

$$\sum_{t=1}^T \log \frac{p_{\theta}(x_{t-1}|x_t)}{q(x_t|x_{t-1})} \right].$$

The paper's equation is essentially this.

22. Why does Equation (5) appear?

Equation (3) is correct, but it isn't the most convenient form for optimization.

So the authors algebraically rearrange the ELBO into:

[

L

$$\begin{aligned} & \mathbb{E}_q \\ & \left[\right. \\ & L_T + \\ & \sum_{t>1} L_{t-1} \\ & + \\ & L_0 \\ & \left. \right]. \\ &] \end{aligned}$$

Specifically:

[

L_T

$$\begin{aligned} & D_{\{KL\}} \\ & \left(\right. \\ & q(x_T|x_0) \\ & | \\ & p(x_T) \\ & \left. \right) \\ &] \end{aligned}$$

[

L_{t-1}

$$\begin{aligned} & D_{\{KL\}} \\ & \left(\right. \\ & q(x_{t-1}|x_t,x_0) \\ & | \end{aligned}$$

$$p_{\theta}(x_{t-1}|x_t)$$

and

[

L_0

$$-\log p_{\theta}(x_0|x_1).$$

This decomposition is extremely important.

23. Understand (L_T)

We have:

[

L_T

$$D_{\text{KL}} \left(q(x_T|x_0) \parallel p(x_T) \right).$$

Remember:

$$[p(x_T) = \mathcal{N}(0, I).]$$

And the forward process produces

$$[q(x_T|x_0).]$$

So this term asks:

Does the final noisy representation produced by the forward process look like the standard Gaussian distribution that we use to start generation?

If (T) is sufficiently large, then

$$[q(x_T|x_0) \approx \mathcal{N}(0, I),]$$

so this term becomes relatively small.

Also, since the forward process is fixed, (L_T) does **not depend on the neural network parameters** in the usual DDPM setup.

24. The most important term: (L_{t-1})

We have:

[

$$L_{t-1}$$

$$D_{\{KL\}} \left(q(x_{t-1}|x_t, x_0) \parallel p_{\theta}(x_{t-1}|x_t) \right)$$

\right).
]

There are two distributions here.

True/forward posterior

[
 $q(x_{t-1}|x_t, x_0)$
]

This tells us:

Given the original clean image (x_0) and the noisy image (x_t), what should the previous less-noisy image (x_{t-1}) look like?

Learned reverse distribution

[
 $p_{\theta}(x_{t-1}|x_t)$.
]

This tells us:

Given only (x_t), what does our neural network think (x_{t-1}) should look like?

Training tries to make these two distributions close.

25. This is the heart of diffusion training

The model learns:

[
 $\boxed{p_{\theta}(x_{t-1}|x_t)}$
 $\approx$
 $q(x_{t-1}|x_t, x_0)$
]
]

during training.

Notice something subtle.

The true distribution has:

[
x_0
]

as an additional condition.

But during generation we don't have (x_0).

So the neural network must learn to approximate the correct reverse transition using only:

[
x_t,t.
]

That is why the model can eventually generate from noise.

26. Why is $(q(x_{t-1}|x_t, x_0))$ tractable?

This is where the Gaussian structure becomes extremely useful.

Because both the forward transitions are Gaussian:

[
 $q(x_t|x_{t-1})$
]

is Gaussian.

And the distribution of (x_t) given (x_0) is also Gaussian:

[
 $q(x_t|x_0)$.
]

Therefore, by Gaussian conditioning, we can derive

[
 $q(x_{t-1}|x_t, x_0)$
]

as another Gaussian.

The paper gives:

[

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$$

```
\mathcal{N}
\left(
\mathbf{x}_{t-1};
\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0),
\tilde{\beta}_t I
\right).
```

This is Equation (6).

27. Understanding the posterior mean

The paper gives

[

$$\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0)$$

```
\frac{
\sqrt{\bar{\alpha}_{t-1}}\beta_t
}{
1-\bar{\alpha}_t
}
\mathbf{x}_0
+
\frac{
\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})
}
```


$$\{ 1 - \bar{\alpha}_t \}$$

$$x_t$$

Look at its structure.

It is a weighted combination of:

$$[x_0]$$

and

$$[x_t]$$

So:

$$[$$

$$\boxed{\}$$

$$\tilde{\mu}_t$$

$$\{ \text{weight}_1 \cdot x_0 + \text{weight}_2 \cdot x_t \}$$

This makes intuitive sense.

We know both:

- the original clean image,
- the current noisy image.

Therefore we can calculate what the previous state (x_{t-1}) should statistically look like.

28. Understanding $\tilde{\beta}_t$

The paper gives:

[

$\tilde{\beta}_t$

$$\frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t} \tilde{\beta}_t$$

Thus:

[

$q(x_{t-1} | x_t, x_0)$

$$\mathcal{N}(x_{t-1}; \tilde{\mu}_t, \tilde{\beta}_t)$$

Again, because the posterior is Gaussian, everything becomes analytically manageable.

29. Why is this useful?

Suppose you have:

```
[  
x_0  
]
```

and you randomly generate

```
[  
x_t.  
]
```

You can mathematically calculate the target distribution:

```
[  
q(x_{t-1}|x_t,x_0).  
]
```

Then the neural network predicts:

```
[  
p_{\theta}(x_{t-1}|x_t).  
]
```

You minimize their KL divergence:

```
[  
D_{KL}  
\left(  
q  
|  
p_{\theta}  
\right).  
]
```

So the training problem becomes a supervised-like problem:

```
[  
\boxed{  
\text{known target distribution}  
\quad\longrightarrow\quad  
\text{network prediction}  
}
```

```
}  
]
```

30. What does "Rao-Blackwellized" mean here?

The paper says:

all KL divergences ... can be calculated ... with closed form expressions instead of high variance Monte Carlo estimates.

This sounds complicated, but the basic idea is simple.

Because both distributions are Gaussian, we don't have to estimate the KL divergence by repeatedly sampling.

For two Gaussians, the KL divergence has a closed-form formula.

Therefore we can directly calculate:

```
[  
D_{KL}  
\left(  
\mathcal{N}(\mu_1, \Sigma_1)  
|  
\mathcal{N}(\mu_2, \Sigma_2)  
\right).  
]
```

This gives lower-variance training.

31. The entire DDPM training process

Now let's put everything together.

Suppose you have a dataset of images.

Step 1 — Sample a clean image

```
[
x_0 \sim q_{\text{data}}(x).
]
```

Step 2 — Randomly select a timestep

```
[
t \sim \text{Uniform}\{1, \dots, T\}.
]
```

Step 3 — Sample Gaussian noise

```
[
\epsilon \sim \mathcal{N}(0, I).
]
```

Step 4 — Directly construct (x_t)

Using:

```
[
```

$\boxed{}$

x_t

```
\sqrt{\bar{\alpha}_t} x_0
+
\sqrt{1 - \bar{\alpha}_t} \epsilon.
]
```

Step 5 — Give (x_t) and (t) to the neural network

```
[
(x_t, t)
\rightarrow
\text{network}.
]
```

Step 6 — Network predicts information needed to reverse the noise

In the original formulation this can be expressed through

$$\begin{bmatrix} \mu_{\theta}(x_t, t) \end{bmatrix}$$

(and potentially covariance).

Step 7 — Compare the learned reverse transition with the true posterior

$$\begin{bmatrix} p_{\theta}(x_{t-1}|x_t) \end{bmatrix}$$

versus

$$\begin{bmatrix} q(x_{t-1}|x_t, x_0). \end{bmatrix}$$

Step 8 — Optimize

Minimize the corresponding variational loss.

32. Then how does generation work?

After training, we no longer have (x_0) .

We start with:

$$\begin{bmatrix} x_T \sim \mathcal{N}(0, I). \end{bmatrix}$$

Then:

$$\begin{bmatrix} x_T \\ \rightarrow \\ x_{T-1} \end{bmatrix}$$

$$\begin{aligned} &\rightarrow x_{T-2} \\ &\rightarrow \vdots \\ &\rightarrow x_0. \end{aligned}$$

At every step:

$$\begin{aligned} &[\\ &x_{t-1} \\ &\sim \\ &p_{\theta}(x_t|x_{t-1}). \\ &] \end{aligned}$$

So the neural network gradually transforms noise into a sample.

33. Training versus generation

This distinction is extremely important.

	Training	Generation
Starting point	(x_0) , real data	(x_T) , random noise
Forward process	Used	Not used
Reverse process	Learned	Used
(x_0) available?	Yes	No
Network	Learns denoising	Performs denoising
Direction	$(x_0 \text{ to } x_t)$	$(x_T \text{ to } x_0)$

34. The surprising part: the forward process is easy

The forward process:

```
[  
q(x_t|x_{t-1})  
]
```

is completely known.

You don't need to train it.

You choose:

```
[  
\beta_1,\ldots,\beta_T.  
]
```

Then you can generate arbitrary noisy versions of your training images.

The difficult problem is:

```
[  
\boxed{  
\text{How do we reverse the noise?}  
}  
]
```

The neural network learns that.

35. Why Gaussian distributions?

The paper says:

When the diffusion consists of small amounts of Gaussian noise, it is sufficient to set the sampling chain transitions to conditional Gaussians too.

The important intuition is that if every forward step adds only a **small amount of Gaussian noise**, then reversing one small step can also be well approximated by a Gaussian conditional.

Thus:

$$[q(x_t|x_{t-1})]$$

is Gaussian and we model

$$[p_{\theta}(x_{t-1}|x_t)]$$

as Gaussian.

This makes the mathematics tractable.

36. Why use many small steps?

Imagine trying to transform an image directly into noise in one huge step.

That would be difficult to reverse.

Instead:

$$[\begin{array}{l} \text{image} \\ \rightarrow \\ \text{almost image} \\ \rightarrow \\ \text{slightly noisy image} \\ \rightarrow \\ \vdots \end{array}]$$

Each step is small.

Likewise, the reverse process only needs to solve many small denoising problems.

Conceptually:

$$[\boxed{\text{hard transformation}}]$$

```
\approx
\text{many easy transformations}
}
]
```

37. The connection to VAEs

Since you have been studying VAEs, this comparison is useful.

VAE

Usually:

```
[
x\rightarrow z\rightarrow x.
]
```

There is a relatively low-dimensional latent variable (z).

The encoder learns:

```
[
q_\phi(z|x).
]
```

The decoder learns:

```
[
p_\theta(x|z).
]
```

DDPM

Instead:

```
[
x_0\rightarrow x_1\rightarrow x_2\rightarrow\cdots\rightarrow x_T.
]
```

The latent variables have the **same dimensionality as the data**.

There is a whole chain of latent variables.

The forward process is:

$$[q(x_{1:T}|x_0).$$

The reverse model is:

$$[p_{\theta}(x_{0:T}).$$

So DDPM can be viewed as a special kind of **deep latent-variable model with a very long chain of latent variables**.

38. Connection to the ELBO you were asking about earlier

You previously encountered the VAE ELBO:

$$[$$
$$\mathbb{E}_{q_{\phi}(z|x)}$$
$$\log p_{\theta}(x|z)]$$
$$-D_{KL}(q_{\phi}(z|x)||p(z)).$$
$$]$$

DDPM is also using variational inference.

But its ELBO is decomposed differently.

The DDPM objective can be written approximately as:

[

L

L_T

+

L_{T-1}

+ \cdots +

L_1

+

L_0.

]

Where:

[

L_T

D_{KL}(q(x_T|x_0)||p(x_T))

]

and for intermediate steps:

[

L_{t-1}

D_{KL}

(q(x_{t-1}|x_t,x_0)

|

p_{\theta}(x_{t-1}|x_t)).

]

And the final reconstruction term is:

$$[\text{L}_0 = -\log p_{\theta}(x_0|x_1).]$$

So there is a very strong conceptual connection to the VAE ELBO.

39. But DDPM training is often presented differently

The original DDPM paper discovers a particularly useful parameterization where the network predicts the noise:

$$[\epsilon_{\theta}(x_t, t).]$$

Recall:

[

x_t

$$[\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon.]$$

Therefore, if the network can predict

$$[\epsilon,]$$

it can infer information about the original clean image.

From the equation:

[

x_t

$$\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon,$$

]

we can rearrange:

[

$\boxed{}$

x_0

$$\frac{x_t - \sqrt{1-\bar{\alpha}_t}\epsilon}{\sqrt{\bar{\alpha}_t}}$$

}

}

]

Therefore if

$$|\epsilon_{\theta(x_t,t)}| \approx \epsilon,$$

]

we can estimate:

[

$\hat{x}_0$

$$\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_{\theta}(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

This is the basis of the famous **noise-prediction objective**.

40. The simplified DDPM loss

The practical training objective often becomes:

[

$\boxed{}$

L_{simple}

$$\mathbb{E}_{x_0, t, \epsilon} \left[|\epsilon - \epsilon_{\theta}(x_t, t)|^2 \right],$$

where

[

x_t

$\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon.$
]

So training becomes remarkably simple:

Add known Gaussian noise to a clean image, then train the neural network to predict the noise that you added.

This is why diffusion-model training can look almost like ordinary supervised learning.

41. Very important distinction: ELBO vs simplified loss

This is a point that often causes confusion.

The paper begins with a **variational lower/upper bound formulation**:

[
 L_{VLB} .
]

Then, under a particular parameterization and weighting, one can use the simplified objective:

[

L_{simple}

$|\epsilon - \epsilon_{\theta}(x_t, t)|^2.$
]

So:

[
 $\boxed{}$


```

\text{ELBO}
\rightarrow
\text{Gaussian KL decomposition}
\rightarrow
\text{noise-prediction parameterization}
\rightarrow
\text{simple MSE loss}
}
]

```

The simplified MSE is **not simply the original ELBO written in an identical form**; it is a particular simplified training objective derived from/related to the variational objective.

42. What the network is actually learning

This is perhaps the most useful mental model.

Suppose:

```

[
x_0=\text{clean cat}.
]

```

We select ($t=500$).

We create:

```

[

```

x_{500}

```

\sqrt{\bar{\alpha}_{500}}x_0
+
\sqrt{1-\bar{\alpha}_{500}}\epsilon.
]

```

The network sees:

```
[  
x_{500},500.  
]
```

It **does not see** (x_0).

It tries to answer:

What noise was added to this image?

It predicts:

```
[  
\epsilon_\theta(x_{500},500).  
]
```

The target is the actual sampled:

```
[  
\epsilon.  
]
```

So:

```
[  
\epsilon_\theta(x_{500},500)  
\approx  
\epsilon.  
]
```

Once it knows the noise, it knows how to move (x_{500}) toward a less noisy state.

43. Why can noise prediction generate images?

At first this seems strange.

You might ask:

If the network only learns to predict noise, how does it learn what a cat looks like?

Because the noise is being predicted **conditioned on the noisy image**.

The network must understand the structure of the noisy image to distinguish:

[
 \text{actual signal}
]

from

[
 \text{noise}.
]

Therefore it implicitly learns the structure of the data distribution.

For example, given a very noisy image, it learns statistical structures corresponding to:

- edges,
- textures,
- shapes,
- objects,
- spatial relationships,
- etc.

44. Score matching connection

The paper also says the parameterization reveals an equivalence with **denoising score matching**.

The score of a distribution is:

[
 \nabla_x \log p(x).
]

It tells us roughly:

In which direction should we move (x) to reach regions of higher probability density?

For a noisy image distribution, the score tells us how to move toward more probable/cleaner samples.

Diffusion models can therefore be interpreted as learning the score of the data distribution at many noise levels:

$$\left[\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t) \right]$$

Then sampling can be related to **annealed Langevin dynamics**.

This is one of the deeper theoretical connections of the paper.

45. The complete conceptual picture

You can now view DDPM as follows:

$$\left[\boxed{\text{DATA}} \right]$$

↓

Forward diffusion

$$\left[\begin{array}{c} \mathbf{x}_0 \\ \rightarrow \\ \mathbf{x}_1 \\ \rightarrow \\ \mathbf{x}_2 \\ \rightarrow \\ \vdots \\ \rightarrow \\ \mathbf{x}_T \end{array} \right]$$

↓

$$\left[\boxed{\mathbf{x}_T \approx \mathcal{N}(\mathbf{0}, \mathbf{I})} \right]$$

Then learn the reverse:

```
[  
  \boxed{  
    x_T  
  }  
]
```

↓

```
[  
  p_{\theta(x_{T-1}|x_T)}  
]
```

↓

```
[  
  p_{\theta(x_{T-2}|x_{T-1})}  
]
```

↓

```
[  
  \vdots  
]
```

↓

```
[  
  p_{\theta(x_0|x_1)}  
]
```

↓

```
[  
  \boxed{  
    \text{GENERATED DATA}  
  }  
]
```

46. Equation-by-equation summary

Equation (1): Reverse process

[

$$p_{\theta}(x_{0:T})$$

$$p(x_T) \prod_{t=1}^T p_{\theta}(x_{t-1}|x_t)$$

Means:

Start from Gaussian noise and repeatedly apply learned denoising transitions.

Equation (2): Forward process

[

$$q(x_{1:T}|x_0)$$

$$\prod_{t=1}^T q(x_t|x_{t-1})$$

with

[

$$q(x_t|x_{t-1})$$

$\mathcal{N}$
(

$$x_t; \\ \sqrt{1-\beta_t}x_{t-1}, \\ \beta_t \\). \\]$$

Means:

Gradually destroy the data using known Gaussian noise.

Equation (3): Variational objective

$$[\\ -\log p_{\theta}(x_0) \\ \leq L. \\]$$

Means:

Instead of directly maximizing the likelihood, optimize a tractable variational bound.

Equation (4): Closed-form noising

$$[$$

$$q(x_t|x_0)$$

$$\mathcal{N} \\ (\\ x_t; \\ \sqrt{\bar{\alpha}_t}x_0, \\ (1-\bar{\alpha}_t)I \\). \\]$$

Equivalent sampling equation:

[

$\boxed{}$

x_t

$$\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon.$$

Means:

Jump directly from clean data to any noise level.

Equation (5): ELBO decomposition

[

L

$$L_T + \sum_{t>1} L_{t-1} + L_0.$$

The terms are:

$$L_T = D_{\text{KL}}(q(x_T|x_0)||p(x_T))$$

[

$L_{\{t-1\}}$

$D_{\{KL\}}$
 $(q(x_{\{t-1\}}|x_t, x_0)$
 $|$
 $p_{\theta}(x_{\{t-1\}}|x_t)$
 $]$

[
 $L_0 = -\log p_{\theta}(x_0|x_1).$
 $]$

Means:

The overall objective breaks into a collection of Gaussian matching problems plus a final reconstruction term.

Equation (6): True reverse posterior

[

$q(x_{\{t-1\}}|x_t, x_0)$

$\mathcal{N}$
 $($
 $x_{\{t-1\}};$
 $\tilde{\mu}_t,$
 $\tilde{\beta}_{t-1}$
 $).$
 $]$

Means:

Given the clean image and current noisy image, we can analytically determine the distribution of the previous state.

Equation (7): Posterior parameters

[

$\tilde{\mu}_t$

$$\frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t]$$

and

[

$\tilde{\beta}_t$

$$\frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t.]$$

Means:

Because everything is Gaussian, the true posterior mean and variance can be calculated exactly.

47. One simple numerical intuition

Suppose, just for intuition, that at some timestep:

[
 $\bar{\alpha}_t = 0.25$.
]

Then:

[
 $\sqrt{\bar{\alpha}_t} = 0.5$
]

and

[

$\sqrt{1 - \bar{\alpha}_t}$

$\sqrt{0.75}$
 ≈ 0.866 .
]

Therefore:

[

x_t

$0.5x_0 + 0.866\epsilon$.

]

So the noisy sample contains:

[

$\boxed{50\% \text{ signal coefficient}}$

]

and

[

$\boxed{86.6\% \text{ noise coefficient}}$

]

in this particular parameterization.

As t increases, the signal coefficient decreases and the noise coefficient increases.

Eventually:

[

$x_T \approx \epsilon$.

]

48. The deepest idea in one sentence

The entire DDPM idea can be summarized as:

[

$\boxed{\text{Define an easy process that turns data into Gaussian noise, then learn its reverse.}}$

]

And the mathematical trick that makes this practical is:

[

$\boxed{x_t =$

$\sqrt{\alpha_t}x_0 +$

$\sqrt{1-\alpha_t}\epsilon$

```
}
]
```

because this lets us create **any noise level directly**.

Then the network learns:

```
[
\boxed{
\epsilon_{\theta(x_t,t)} \approx \epsilon
}
]
```

which gives us a way to reverse the corruption.

The complete chain you should keep in your head

```
[
\boxed{
x_0
\rightarrow [\text{known}] \{\text{add Gaussian noise}\}
x_t
}
]
```

during training:

```
[
\boxed{
x_{t,t}
\rightarrow \{\text{neural network}\}
\epsilon_{\theta}
}
]
```

and therefore:

```
[
\boxed{
\epsilon_{\theta}
\rightarrow
\hat{x}_0
\rightarrow
```

```
\text{denoised } x_{t-1}
}
]
```

during generation:

```
[
\boxed{
\mathcal{N}(0,I)
\rightarrow
x_{T-1}
\rightarrow
x_{T-2}
\rightarrow
\cdots
\rightarrow
x_0.
}
]
```

The most important conceptual distinction is therefore:

Forward diffusion is a fixed, known corruption process. Reverse diffusion is the learned generative process. Variational inference provides the training objective that makes the learned reverse process approximate the true reverse of the corruption process.

A Markov chain is a mathematical model for a system that moves between different states, where the probability of the next state depends only on the current state, not on the complete history.

1. Basic idea

Suppose the weather can be:

-  Sunny
-  Rainy

We might define transition probabilities:

$P(\text{Sunny} \rightarrow \text{Sunny})=0.8$ $P(\text{Sunny} \rightarrow \text{Rainy})=0.2$ $P(\text{Rainy} \rightarrow \text{Sunny})=0.4$ $P(\text{Rainy} \rightarrow \text{Rainy})=0.6$

So, if today is Sunny, tomorrow has an 80% chance of being Sunny and a 20% chance of being Rainy.

2. Markov property

The defining assumption is:

$$P(X_{t+1} | X_t, X_{t-1}, \dots, X_0) = P(X_{t+1} | X_t)$$

In words:

The future depends on the present state, not on the past states.

For example, if today's weather is Sunny, the probability of tomorrow's weather does not depend on whether the last 5 days were rainy or sunny.

3. Transition matrix

We can represent all transition probabilities using a matrix:

$$P = \begin{bmatrix} 0.80 & 0.40 \\ 0.20 & 0.6 \end{bmatrix}$$

where rows represent the current state and columns represent the next state.

Each row sums to 1:

$$0.8 + 0.2 = 1 \quad 0.4 + 0.6 = 1$$

If the current probability distribution is

$$p_t = [0.70 \ 0.3],$$

then the distribution at the next time step is

$$p_{t+1} = p_t P.$$

4. Why Markov chains matter for diffusion models

This connects directly to the diffusion models material you were reading.

A diffusion process can be defined as a Markov chain:

$$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T$$

where each step adds a small amount of noise:

$$q(x_t | x_{t-1}).$$

The Markov property means:

$$q(x_t | x_{t-1}, x_{t-2}, \dots, x_0) = q(x_t | x_{t-1}).$$

So, to generate x_t , we only need x_{t-1} , rather than the entire sequence $x_0, \dots, x_{t-2}$.

In diffusion models, the forward process is therefore a Markov chain that gradually destroys information by adding noise:

$$x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T$$

and the neural network learns the reverse process:

$$x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_1 \rightarrow x_0.$$

That is the key reason the DDPM paper describes diffusion models as parameterized Markov chains.

Yes — you're asking where the $(\sqrt{1-\beta_t})$ actually comes from, rather than just how to use it.

It comes directly from choosing the mean and variance of the Gaussian transition.

1. Start with the desired forward diffusion step

We want to gradually add noise to (x_{t-1}) . Define:

$$\begin{aligned} &[\\ &x_t = a_t x_{t-1} + b_t \epsilon, \\ &\quad \epsilon \sim \mathcal{N}(0, I). \\ &] \end{aligned}$$

Because (ϵ) is Gaussian,

$$\begin{aligned} &[\\ &b_t \epsilon \sim \mathcal{N}(0, b_t^2 I). \\ &] \end{aligned}$$

Therefore,

$$\begin{aligned} &[\\ &x_t \mid x_{t-1} \\ &\sim \\ &\mathcal{N}(a_t x_{t-1}, b_t^2 I). \\ &] \end{aligned}$$

So the coefficients (a_t) and (b_t) determine the mean and variance.

2. Now introduce (β_t)

DDPM defines the noise schedule so that

[
 $\boxed{\text{variance added at step } t = \beta_t}$
]

Therefore we need

[
 $b_t^2 = \beta_t$.
]

Hence,

[
 $\boxed{b_t = \sqrt{\beta_t}}$.
]

So the noise part becomes

[
 $\sqrt{\beta_t} \epsilon$.
]

3. Why $(1 - \beta_t)$ for the signal?

We want the total scale of the signal and noise to be normalized.

So DDPM chooses

[
 $a_t^2 + b_t^2 = 1$.
]

Since

[
 $b_t^2 = \beta_t$,
]

we get

[
 $a_t^2 + \beta_t = 1$.
]

Therefore,

$$[a_t^2 = 1 - \beta_t]$$

and hence

$$[\boxed{a_t = \sqrt{1 - \beta_t}}.]$$

Thus:

$$[\boxed{x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon}]$$

and equivalently,

[

$\boxed{\phantom{x_t = \sqrt{1 - \beta_t} x_{t-1} + \sqrt{\beta_t} \epsilon}}$

$q(x_t | x_{t-1})$

$$\begin{aligned} & \mathcal{N} \\ & \left(\right. \\ & x_t; \\ & \left. \sqrt{1 - \beta_t} x_{t-1}, \right. \\ & \left. \beta_t \right) \\ & \left. \right) \\ & \end{aligned}$$

The key point

The square roots are there because variance is the square of the standard deviation.

$$\left[\begin{array}{l} \underbrace{\sqrt{\beta_t}}_{\text{standard deviation}} \\ \quad \Longrightarrow \quad \\ \underbrace{\beta_t}_{\text{variance}} \end{array} \right]$$

and similarly,

$$\underbrace{\sqrt{1-\beta_t}}_{\text{signal scaling}} \quad \Longrightarrow \quad \underbrace{1-\beta_t}_{\text{signal power/variance contribution}}.$$

So $(\sqrt{1-\beta_t})$ isn't randomly chosen. It is the coefficient whose square gives the desired $(1-\beta_t)$ scaling.

Exactly — this is the important step to understand. The equation

$$\begin{bmatrix} a_t^2 + b_t^2 = 1 \end{bmatrix}$$

is not a mathematical necessity of every Gaussian transition. It is a design choice in the DDPM forward process.

Let's derive why.

1. Start with the general form

Suppose we define the diffusion step as

$$x_t = a_t x_{t-1} + b_t \epsilon_t$$

where

$$[\epsilon \sim \mathcal{N}(0, I)]$$

We want (x_t) to contain:

- some amount of the previous signal (x_{t-1})
- some amount of new noise (ϵ)

Now ask: how should we choose (a_t) and (b_t) ?

2. We want the overall scale to remain controlled

Imagine we didn't impose any constraint.

For example, suppose

```
[  
a_t=1.2,\quad b_t=0.8.  
]
```

Then

```
[  
x_t=1.2x_{t-1}+0.8\epsilon.  
]
```

The signal is being amplified by (1.2), while noise is also being added.

After many diffusion steps, the magnitude/variance of (x_t) could grow uncontrollably.

We instead want the total variance/power to stay normalized.

For independent components, the variances add:

```
[
```

$\text{Var}(a_t x_{t-1} + b_t \epsilon)$

```
a_t^2 \text{Var}(x_{t-1})  
+  
b_t^2 \text{Var}(\epsilon).  
]
```

If both are normalized to variance (1), then

[

$\text{Var}(x_t)$

$a_t^2 + b_t^2$.
]

To keep the resulting variance equal to (1), we choose

[
 $a_t^2 + b_t^2 = 1$.
]

That's the reason.

3. Now introduce (β_t)

DDPM chooses the amount of new noise variance to be

[
 $b_t^2 = \beta_t$.
]

Therefore,

[
 $a_t^2 + \beta_t = 1$.
]

So

[
 $a_t^2 = 1 - \beta_t$
]

and therefore

```
[
a_t=\sqrt{1-\beta_t}.
]
```

Thus:

```
[
\boxed{
x_t=
\sqrt{1-\beta_t}x_{t-1}
+
\sqrt{\beta_t}\epsilon
}
]
```

One subtle but important point

Don't think:

"Because it's Gaussian, $(a_t^2+b_t^2=1)$."

That's not true.

Rather:

DDPM chooses the coefficients this way so that the signal/noise scaling is normalized and the variance remains controlled.

The Gaussian assumption tells us that

```
[
\epsilon\sim\mathcal{N}(0,I),
]
```

while the particular coefficients

```
[
\sqrt{1-\beta_t},\quad\sqrt{\beta_t}
]
```

come from the chosen diffusion design.

Yes — you're very close, but there is one important distinction.

When deriving

[

$q(x_t \mid x_{t-1})$

$\mathcal{N}(\text{left}(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I) \mid \text{right}),$
]

we condition on (x_{t-1}) . That means (x_{t-1}) is treated as a fixed/known value for this particular transition.

So, from the perspective of this conditional distribution:

[
 $x_{t-1} = \text{fixed}$
]

and the only randomness comes from

[
 $\epsilon \sim \mathcal{N}(0, I).$
]

Therefore,

[

x_t

$\sqrt{1-\beta_t}x_{t-1}$
+
 $\sqrt{\beta_t}\epsilon.$
]

But here's the subtle part

You said:

"doesn't consider its noise"

For this particular conditional transition, yes, we don't separately consider the randomness of (x_{t-1}) .

We are asking:

Given this particular (x_{t-1}) , what distribution does (x_t) have?

So:

[
 $x_{t-1} \propto \boxed{\text{fixed}}$
]

and

[
 $\epsilon \propto \boxed{\text{random}}$.
]

Hence:

[
 $\boxed{x_t \mid x_{t-1}}$
 $\sim$
 $\mathcal{N}$
 $\left(\sqrt{1-\beta_t}x_{t-1}, \beta_t\right)$
]
]

However, (x_{t-1}) itself DOES contain noise

This is very important in diffusion.

Earlier we had:

[

x_{t-1}

$\sqrt{1-\beta_{t-1}}x_{t-2}$
+
 $\sqrt{\beta_{t-1}}\epsilon_{t-1}.$
]

So (x_{t-1}) already contains noise from previous steps.

We're simply not explicitly expanding that noise when considering

[
 $q(x_t \mid x_{t-1}).$
]

If we expand everything back to (x_0) , then all the previous noise terms appear.

That's why you can think of it as:

[
 $\boxed{\text{At each step, treat the current } x_{t-1} \text{ as given and add new noise.}}$
]
]

This distinction between conditional randomness and the fact that (x_{t-1}) was itself generated randomly is fundamental to understanding diffusion models.

Yes — exactly. A weighted average of numbers can absolutely be negative even if all the weights are positive.

So your objection is correct:

If $(\log(q/p))$ can be negative, and $(q(x))$ is positive, why can't
 $(\int q(x) \log(q(x)/p(x)) dx)$ be negative?

The answer is: because the values of $(\log(q/p))$ and the weights (q) are not arbitrary. They come from two probability distributions, and that imposes a special mathematical constraint.

The key trick

Let

$$r(x) = \frac{p(x)}{q(x)}.$$

Then

$$D_{\mathrm{KL}}(q|p)$$

$$\mathbb{E}_q \left[\log \frac{q}{p} \right]$$

$$-\mathbb{E}_q [\log r].$$

Now use the fact that

$$\boxed{\log r \leq r-1}$$

for every $(r>0)$.

Therefore,

$$-\log r \geq 1-r.$$

Taking expectation under (q) :

$$\begin{aligned} -\mathbb{E}_q [\log r] \\ \geq \\ 1-\mathbb{E}_q [r]. \end{aligned}$$

But

[

$$\mathbb{E}_q[r]$$

$$\int q(x)\frac{p(x)}{q(x)}dx$$

$$\int p(x)dx$$

1.

]

Therefore,

[

$$D_{\mathrm{KL}}(q|p)$$

$$\geq$$

$$1-1$$

1.

]

Hence

[

$$\boxed{D_{\mathrm{KL}}(q|p)\geq 0}.$$

]

So your intuition is right

If I simply gave you arbitrary numbers like

[
-5,\quad -2,\quad +1
]

with positive weights, their weighted average could certainly be negative.

KL divergence is special because (q) and (p) are normalized probability distributions.

That's what prevents the overall expectation from becoming negative.

And importantly:

[
 $\log(q/p)$ itself is NOT always positive
]

Only its expected value under (q) has the guaranteed non-negative property.

It means the neural network is being asked to figure out how to go one step backward in the diffusion process.

Let's connect it to what we just discussed.

1. Forward process

During forward diffusion:

[
 $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \cdots \rightarrow x_T$
]

we progressively add noise.

For example:

[
 $x_t =$
 $\sqrt{1-\beta_t}x_{t-1}$
+
 $\sqrt{\beta_t}\epsilon.$
]

At large (t), (x_t) is very noisy.

2. Now we want to go backward

We want:

$$[x_t \rightarrow x_{t-1}.]$$

The true reverse distribution is

$$[q(x_{t-1}|x_t, x_0).]$$

But during generation, we don't know (x_0) . We only have (x_t) .

So we learn a neural network:

$$[\boxed{x_t, t} \rightarrow \text{Neural Network}]$$

The network sees:

- (x_t) : the current noisy image
- (t) : how much noise has been added

and produces information that helps us estimate the previous state.

3. What is $(\mu_\theta(x_t, t))$?

Suppose we decide that the reverse transition is Gaussian:

$$[$$

$$p_\theta(x_{t-1}|x_t)$$

$\mathcal{N}$
 $\left($

$$x_{t-1};$$

$$\mu_{\theta}(x_t, t),$$

$$\Sigma_{\theta}(x_t, t)$$

$$\right).$$

$$]$$

Here,

$$[\mu_{\theta}(x_t, t)]$$

is the mean predicted by the neural network.

It essentially says:

"Given this noisy image (x_t) at timestep (t), where do I think the previous image (x_{t-1}) should be centered?"

Then we sample around that predicted mean.

For example:

$$[$$

$$x_{t-1}$$

$$\mu_{\theta}(x_t, t)$$

$$+$$

$$\text{appropriate noise}.$$

$$]$$

4. Why does the network need (t)?

Consider two noisy images:

$$[x_{50}]$$

and

[
 x_{900} .
]

They may both look noisy, but they require different amounts of denoising.

At (t=50):

[
 $\text{only a little noise}$
]

At (t=900):

[
 a lot of noise .
]

So the network needs to know which stage of the diffusion process it is at.

That's why:

[
 $\boxed{(x_t, t) \rightarrow \mu_\theta(x_t, t)}$
]

rather than simply

[
 $x_t \rightarrow \mu_\theta(x_t)$.
]

5. What does "and potentially covariance" mean?

A Gaussian has two important parameters:

[
 $\mathcal{N}(\mu, \Sigma)$.
]

So theoretically the network could predict both:

[
 $\mu_\theta(x_t, t)$
]

and

```
[  
\Sigma_\theta(x_t,t).  
]
```

That would mean:

Mean: Where should (x_{t-1}) be?

Covariance: How uncertain are we about (x_{t-1}) ?

But in the original DDPM formulation, the variance is generally fixed according to the noise schedule, so the neural network primarily learns the information needed to determine the mean.

The whole idea

```
[  
\boxed{  
x_t  
\overset{\text{network}}{\longrightarrow}  
\text{information about how to denoise}  
\longrightarrow  
x_{t-1}  
}  
]
```

And in the original formulation this can be represented as learning

```
[  
\boxed{\mu_\theta(x_t,t).  
]
```

One subtle point: in modern diffusion explanations, you'll often hear "the network predicts the noise (ϵ)" rather than "the network predicts the mean." These are two closely related parameterizations. The network can predict $(\epsilon_\theta(x_t,t))$, and from that prediction we can compute the reverse mean $(\mu_\theta(x_t,t))$.

Yes — conceptually, that's correct, with one small qualification.

In the original DDPM formulation:

```
[  
(x_t,t)\rightarrow \text{neural network}  
]
```


The network predicts the mean of the reverse Gaussian:

$$\begin{bmatrix} \mu_{\theta}(x_t, t) \end{bmatrix}$$

Then we sample (x_{t-1}) around that mean:

[

$\boxed{\mu_{\theta}(x_t, t) + \sigma_t \epsilon}$

x_{t-1}

$$\begin{bmatrix} \mu_{\theta}(x_t, t) \\ + \\ \sigma_t \epsilon \end{bmatrix}$$

where

$$\begin{bmatrix} \epsilon \sim \mathcal{N}(0, I) \end{bmatrix}$$

So the process is:

$$\begin{bmatrix} \boxed{x_t} \\ \rightarrow \text{NN} \\ \rightarrow \mu_{\theta} \\ \rightarrow \mu_{\theta} + \text{Gaussian noise} \\ \rightarrow x_{t-1} \end{bmatrix}$$

```
}  
]
```

But why add noise?

Because the reverse transition is a distribution, not a single fixed value:

```
[
```

$p_{\theta}(x_{t-1}|x_t)$

```
\mathcal{N}(\mu_{\theta}, \sigma_t^2).  
]
```

The neural network gives the center of this distribution, and the Gaussian noise lets us sample from around that center.

For example, imagine the network says:

```
[  
\mu_{\theta}=5  
]
```

and the standard deviation is (1).

Then possible samples might be:

```
[  
4.7, \quad 5.2, \quad 5.8, \quad 4.1, \ldots  
]
```

They are all possible (x_{t-1}) .

One important modern detail

In many implementations, the network doesn't directly output (μ_{θ}) . Instead, it predicts the noise:

```
[  
\epsilon_{\theta}(x_t, t).  
]
```

Then we use that predicted noise to calculate the mean (μ_{θ}) .

So both descriptions are valid:

```
[  
\boxed{\text{NN predicts mean}}  
]
```

or, in the common DDPM parameterization,

```
[  
\boxed{\text{NN predicts noise } \epsilon_\theta  
\text{, from which mean is calculated}.}  
]
```

Yes. Your summary is essentially correct. The Rao–Blackwellized part is worth clarifying, because it is easy to misunderstand.

30. What does "Rao-Blackwellized" mean here?

The basic problem is that the DDPM objective contains terms such as

```
[  
D_{\mathrm{KL}}  
\left(  
q(x_{t-1} \mid x_t, x_0)  
; ;  
p_\theta(x_{t-1} \mid x_t)  
\right).  
]
```

At first, you might think:

"To calculate this KL, I need to sample (x_{t-1}) many times and estimate the expectation."

That would be a Monte Carlo estimate.

But we already know something very useful:

```
[  
q(x_{t-1} \mid x_t, x_0)  
]
```

is Gaussian, and

[
 $p_{\theta}(x_{t-1} \mid x_t)$
]

is also Gaussian.

Therefore, instead of sampling, we can use the analytical Gaussian KL formula.

For example, in the general multivariate case,

[
 D_{KL}
 $\left(\mathcal{N}(\mu_1, \Sigma_1) \right.$
 $\left. \middle| \mathcal{N}(\mu_2, \Sigma_2) \right)$
]

has a closed-form expression:

[
 $\frac{1}{2}$
 $\left[\text{tr}(\Sigma_2^{-1} \Sigma_1) + \right.$
 $+$
 $(\mu_2 - \mu_1)^T \Sigma_2^{-1} (\mu_2 - \mu_1)$
 $\left. - k + \log \frac{|\Sigma_2|}{|\Sigma_1|} \right]$
]

So we can calculate the KL directly.

Where does "Rao-Blackwellized" come in?

The general idea of Rao-Blackwellization is:

Instead of estimating something using a noisy random sample, condition on information that we already know and calculate the conditional expectation analytically when possible.

That reduces variance.

In this DDPM setting, because the relevant conditional distributions are Gaussian, we can analytically evaluate the expectation/KL rather than relying on noisy Monte Carlo sampling.

So your intuition is right:

```
[
\boxed{
\text{analytic Gaussian calculation}
\quad\rightarrow\quad
\text{less Monte Carlo noise}
}
]
```

But strictly speaking, "Rao–Blackwellized" is a variance-reduction idea, not simply a synonym for "Gaussian KL has a closed form."

31. Entire DDPM training process

Your sequence is good. Let me make one correction to the notation in Step 4.

Define

```
[
\alpha_t=1-\beta_t
]
```

and

```
[
\bar{\alpha}_t=\prod_{s=1}^t\alpha_s.
]
```

Then the direct sampling equation is

```
[
\boxed{
x_t=
\sqrt{\bar{\alpha}_t}x_0+
\sqrt{1-\bar{\alpha}_t}\epsilon
}
]
```

where

```
[  
\epsilon \sim \mathcal{N}(0, I).  
]
```

So the complete process is:

Step 1 — Get a clean image

```
[  
x_0 \sim q_{\{\text{data}\}}(x).  
]
```

Step 2 — Randomly choose a timestep

```
[  
t \sim \operatorname{Uniform}\{1, \ldots, T\}.  
]
```

Step 3 — Generate Gaussian noise

```
[  
\epsilon \sim \mathcal{N}(0, I).  
]
```

Step 4 — Create the noisy image directly

```
[  
\boxed{  
x_t =  
\sqrt{\bar{\alpha}_t} x_0 +  
\sqrt{1 - \bar{\alpha}_t} \epsilon  
}  
]
```

This is important: we don't have to actually perform all (t) forward steps. We can directly construct (x_t) from (x_0) .

Step 5 — Give (x_t) and (t) to the network

```
[  
(x_t, t) \rightarrow \text{Neural Network}.  
]
```

Step 6 — Network predicts how to reverse the noise

In the original formulation, this can be expressed as predicting the reverse Gaussian mean:

$$\begin{bmatrix} \mu_{\theta}(x_t, t) \end{bmatrix}$$

So:

[

$$p_{\theta}(x_{t-1}|x_t)$$

$$\mathcal{N} \left(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t) \right)$$

In the commonly used noise-prediction parameterization, however, the network predicts

$$\begin{bmatrix} \epsilon_{\theta}(x_t, t) \end{bmatrix}$$

and this prediction is then used to obtain the corresponding reverse mean.

Step 7 — Compare with the true posterior

Because during training we know (x_0) , we can calculate

$$\begin{bmatrix} q(x_{t-1}|x_t, x_0) \end{bmatrix}$$

And from the model we have

$$p_{\theta}(x_{t-1}|x_t).$$

Therefore we minimize

$$\boxed{D_{\mathrm{KL}}\left[q(x_{t-1}|x_t, x_0) \parallel p_{\theta}(x_{t-1}|x_t)\right]}.$$

Step 8 — Train the network

The network learns parameters θ such that

$$p_{\theta}(x_{t-1}|x_t)$$

becomes close to the true reverse posterior.

The big picture

The whole DDPM idea can therefore be summarized as:

$$\boxed{\begin{array}{l} \text{Clean } x_0 \\ \xrightarrow{\text{add noise}} x_t \\ \xrightarrow{\text{NN}} \text{denoising information} \\ \xrightarrow{\text{reverse Gaussian}} x_{t-1} \end{array}}$$

and this is repeated during generation:

[
 $x_T \rightarrow x_{T-1} \rightarrow \dots \rightarrow x_1 \rightarrow x_0$.
]

One especially important distinction: during training, (x_0) is known, so we can construct the true posterior $(q(x_{t-1}|x_t, x_0))$. During generation, (x_0) is unknown, so we only have (x_t) and the neural network's prediction.

Yes — in the commonly used DDPM training formulation, the neural network outputs a prediction of the noise.

Specifically:

[
 $\epsilon_\theta(x_t, t)$
]

The network receives:

[
 (x_t, t)
]

and outputs:

[
 $\text{predicted noise } \epsilon_\theta$
]

Why predict noise?

We created (x_t) during training using

[
 $x_t =$
 $\sqrt{\alpha_t} x_0 +$
 $\sqrt{1 - \alpha_t} \epsilon$.
]

We know the actual noise (ϵ) because we generated it ourselves.

So training becomes very simple:

$$\boxed{\epsilon_{\theta}(x_t, t) \approx \epsilon}$$

and the loss is often

$$L = |\epsilon - \epsilon_{\theta}(x_t, t)|^2$$

Then how do we get (x_{t-1}) ?

The network's noise prediction can be converted into the reverse-process mean:

$$\mu_{\theta}(x_t, t)$$

and then we sample:

[

$\boxed{\mu_{\theta}(x_t, t)$

x_{t-1}

$$\mu_{\theta}(x_t, t) + \sigma_t z$$

where

[
 $z \sim \mathcal{N}(0, I)$.
]

So there are two levels to keep separate:

[
 $\boxed{$
 (x_t, t)
 $\rightarrow$
 NN
 $\rightarrow$
 ϵ_θ
 $\rightarrow$
 μ_θ
 $\rightarrow$
 x_{t-1}
 $\}$
]

The neural network does not directly output (x_{t-1}) in the standard noise-prediction formulation.

It predicts the noise contained in (x_t) , and that prediction is used to determine how to denoise (x_t) .

Exactly. This is the key connection in DDPM. Let's derive it directly.

We start with the forward equation:

[

x_t

$\sqrt{\bar{\alpha}_t} x_0$
 $+$
 $\sqrt{1 - \bar{\alpha}_t} \epsilon$.
]

We want the neural network to predict

$$[\epsilon \theta(x_t, t) \approx \epsilon.]$$

1. Rearrange the forward equation

Start with

$$[$$

$$x_t$$

$$\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.]$$

Move the noise term:

$$[$$

$$\sqrt{\bar{\alpha}_t} x_0$$

$$x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon.]$$

Therefore,

$$[$$

$$x_0$$

$$\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}$$

$$\sqrt{\bar{\alpha}_t}$$

$$\}.$$

$$]$$

So if the network predicts (ϵ_θ), we can estimate the original clean image:

[

$\boxed{}$

$\hat{x}_0$

$$\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}$$

$$\}$$

$$\}$$

$$]$$

2. But we actually need the reverse mean

The true posterior is

[

$q(x_{t-1} | x_t, x_0)$

$$\mathcal{N}(x_{t-1}; \tilde{\mu}_t, \tilde{\beta}_t).$$

$$]$$

Its mean is

[

\boxed{

\tilde{\mu}_t

```
\frac{
\sqrt{\bar{\alpha}_{t-1}}\beta_t
}{
1-\bar{\alpha}_t
}x_0
+
\frac{
\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})
}{
1-\bar{\alpha}_t
}x_t
}
```

Now substitute our estimate

```
[
x_0\rightarrow\hat{x}_0.
]
```

That gives the model's reverse mean:

[

\mu_{\theta}(x_t,t)

```
\frac{
\sqrt{\bar{\alpha}_{t-1}}\beta_t
}{
1-\bar{\alpha}_t
}\hat{x}_0
```

$$+ \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t.$$

So technically:

$$\boxed{\epsilon_\theta \rightarrow \hat{x}_0 \rightarrow \mu_\theta}$$

3. There is also a simpler direct formula

After substituting and simplifying, the reverse mean can be written directly in terms of the predicted noise:

$$[$$

$$\boxed{\mu_\theta(x_t, t)}$$

$$\frac{1}{\sqrt{\alpha_t}}$$

$$\left($$

$$\left($$

x_t

```
\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}
\epsilon_{\theta}(x_t,t)
\right)
}
]
```

This is the formula you will usually see in DDPM implementations.

So the complete picture is:

```
[
\boxed{
x_t
\rightarrow \text{NN}
\epsilon_{\theta}
\rightarrow \text{formula}
\mu_{\theta}
\rightarrow + \text{Gaussian noise}
x_{t-1}
}
]
```

The really important intuition is:

The network predicts how much noise is present in (x_t) . Once we know the estimated noise, we can subtract the appropriate amount of noise from (x_t) , which gives us the mean of the reverse distribution.

So the network doesn't magically output the mean. We calculate the mean from its predicted noise.

No. This is exactly where the Markov property is easy to misapply.

The Markov property gives:

[

$\boxed{}$

$$q(x_t \mid x_{t-1}, x_0)$$

$$q(x_t \mid x_{t-1})$$

}

]

It does not give:

[

$$q(x_{t-1} \mid x_t, x_0)$$

$$q(x_{t-1} \mid x_t).$$

]

Why?

The forward Markov chain is

[

$$x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_{t-1} \rightarrow x_t.$$

]

The Markov property says:

Once you know (x_{t-1}) , the earlier history $(x_0, \dots, x_{t-2})$ gives no additional information about the future (x_t) .

So:

[

$$x_0 \rightarrow x_{t-1} \rightarrow x_t$$

]

means

[

$$x_t \perp x_0 \mid x_{t-1}.$$

]

But you're asking about the reverse direction:

[
 $q(x_{t-1} \mid x_t, x_0).$
]

Here, knowing (x_0) does give additional information about (x_{t-1}) .

Simple analogy

Imagine:

[
 $\text{Student's preparation}$
 $\rightarrow$
 knowledge
 $\rightarrow$
 $\text{exam score}.$
]

The Markov property says:

Once I know the student's knowledge, their preparation history doesn't additionally determine the exam score.

But suppose I tell you the exam score and ask:

"What was the student's knowledge?"

Now knowing their preparation history could help you estimate their knowledge.

So:

[

$P(\text{score} \mid \text{knowledge}, \text{preparation})$

$P(\text{score} \mid \text{knowledge})$
]

does not imply

[

$P(\text{knowledge} \mid \text{score}, \text{preparation})$

$P(\text{knowledge} \mid \text{score}).$
]

In diffusion

We have:

[

$q(x_t \mid x_{t-1}, x_0)$

$q(x_t \mid x_{t-1}).$
]

But when we reverse the question,

[
 $q(x_{t-1} \mid x_t, x_0),$
]

the (x_0) information matters.

In fact, Bayes' rule shows exactly why:

[
 $q(x_{t-1} \mid x_t, x_0)$
 $\propto$
 $q(x_t \mid x_{t-1}, x_0)$
 $q(x_{t-1} \mid x_0).$
]

Using the Markov property:

[

$q(x_t \mid x_{t-1})$
 $q(x_{t-1} \mid x_0).$
]

Notice that

[
 $\boxed{q(x_{t-1} \mid x_0)}$
]

is still there.

That's precisely the information about the earlier clean image that helps determine which (x_{t-1}) is plausible.

The key distinction

[

$\boxed{$

$q(x_t \mid x_{t-1}, x_0)$

$q(x_t \mid x_{t-1})$
}
]

Yes — Markov property.

But

[
 $\boxed{$
 $q(x_{t-1} \mid x_t, x_0)$
 $\neq$

$$q(x_{t-1} \mid x_t)$$

in general.

This is why DDPM can use the tractable posterior $q(x_{t-1} \mid x_t, x_0)$ during training, while learning $p_\theta(x_{t-1} \mid x_t)$ for generation.

Yes — that is the right connection. The reason $q(x_{t-1} \mid x_t, x_0)$ appears is fundamentally tied to variational inference and the ELBO.

But one small correction: we are not learning $q(x_{t-1} \mid x_t, x_0)$. It is already known analytically from the forward process. We use it to optimize the model $p_\theta(x_{t-1} \mid x_t)$.

1. What are we actually trying to learn?

The generative/reverse model is

$$[$$

$$p_\theta(x_{0:T})$$

$$p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} \mid x_t).$$

So the thing we ultimately want is

$$[$$

$$\boxed{p_\theta(x_{t-1} \mid x_t)}$$

because this is what we will use during generation:

$$[$$

$$x_T \rightarrow x_{T-1} \rightarrow \cdots \rightarrow x_0.$$

2. Why can't we directly optimize it?

The true posterior of the latent variables,

```
[  
p_{\theta}(x_{1:T} \mid x_0),  
]
```

is difficult to work with.

So we introduce a known distribution:

```
[  
\boxed{q(x_{1:T} \mid x_0)}  
]
```

which is our forward diffusion process.

This is exactly the variational-inference idea.

We use q as the variational distribution in the ELBO.

3. ELBO introduces the KL terms

The ELBO can be decomposed into terms involving

```
[  
D_{\mathrm{KL}}  
\left[  
q(x_{t-1} \mid x_t, x_0)  
|  
p_{\theta}(x_{t-1} \mid x_t)  
\right].  
]
```

So we get:

```
[  
\boxed{  
\text{ELBO optimization}  
\quad \rightarrow \quad  
\text{make } p_{\theta}(x_{t-1} \mid x_t)  
\text{close to }  
q(x_{t-1} \mid x_t, x_0)  
}  
]
```

This is why the "true answer" appears.

4. And why is $q(x_{t-1} \mid x_t, x_0)$ convenient?

Because q was deliberately designed as a Gaussian forward process.

Therefore,

[

$$q(x_{t-1} \mid x_t, x_0)$$

$\mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t I)$
]

can be calculated exactly.

So during training:

[
 $\boxed{q(x_{t-1} \mid x_t, x_0)}$
 $\quad \text{is known}$
]
]

while

[
 $\boxed{p_{\theta}(x_{t-1} \mid x_t)}$
 $\quad \text{is learned.}$
]
]

5. The whole connection

This is the conceptual chain you were looking for:

```
[
\boxed{
\text{Variational inference}
}
]
```

```
[
\downarrow
]
```

```
[
\boxed{\text{ELBO}}
]
```

```
[
\downarrow
]
```

```
[
\boxed{
D_{\mathrm{KL}}
\left[
q(x_{t-1} \mid x_t, x_0)
\mid
p_{\theta}(x_{t-1} \mid x_t)
\right]
}
]
```

```
[
\downarrow
]
```

```
[
\boxed{
\text{learn } p_{\theta}(x_{t-1} \mid x_t)
}
]
```

And because the distributions are Gaussian, this KL can be calculated analytically.

One sentence to remember

DDPM uses the known forward process (q) as the variational distribution; the ELBO then trains the reverse model ($p_{\theta}(x_{t-1} \mid x_t)$) to match the analytically known posterior ($q(x_{t-1} \mid x_t, x_0)$).

So yes: your connection to variational inference and the ELBO is exactly the right way to understand why this conditional distribution appears.

Yes. You're asking an important distinction: what exactly is the ELBO, and why does a KL between those two distributions appear?

1. ELBO is not itself the KL

The ELBO (Evidence Lower Bound) is a lower bound on the log-likelihood:

$$\boxed{\log p_{\theta}(x) \geq \mathrm{ELBO}}$$

For a latent-variable model, one common form is

[

$\boxed{\log p_{\theta}(x, z) - \mathbb{E}_{q(z|x)} [\log p_{\theta}(x, z)]}$

ELBO

$$\mathbb{E}_{q(z|x)} [\log p_{\theta}(x, z) - \log q(z|x)]$$

It can also be written as

[

$\boxed{}$

ELBO

$\mathbb{E}_q[\log p_\theta(x|z)]$

$D_{\mathrm{KL}}(q(z|x)||p(z))$
}
]

So ELBO is an objective/bound, while KL divergence is one mathematical component that appears inside it.

2. In DDPM, the ELBO is decomposed into several KL terms

For diffusion, there are many latent variables:

[
 $x_1, x_2, \dots, x_T$
]

The ELBO decomposition produces terms such as

[
 D_{KL}
 $\left($
 $q(x_{t-1}|x_t, x_0)$
 $|$
 $p_\theta(x_{t-1}|x_t)$
 $\right)$
]

So yes, this KL term comes from the ELBO decomposition.

3. Why are we comparing these two distributions?

Because they represent two different things.

The first:

$$\boxed{q(x_{t-1}|x_t, x_0)}$$

is the true posterior under the forward diffusion process.

During training, we know (x_0) , so this can be calculated.

The second:

$$\boxed{p_{\theta}(x_{t-1}|x_t)}$$

is the reverse distribution that our neural network is learning.

During generation, this is what we actually need because (x_0) won't be available.

Therefore we want:

$$\begin{aligned} & p_{\theta}(x_{t-1}|x_t) \\ & \approx \\ & q(x_{t-1}|x_t, x_0). \end{aligned}$$

The KL measures their difference:

$$D_{\mathrm{KL}}(q|p_{\theta}).$$

4. So your statement is basically correct

You said:

"since both are in KL, that's why we are comparing them."

More precisely:

The ELBO mathematically decomposes into KL divergence terms, and one of those terms compares the known forward posterior ($q(x_{t-1}|x_t, x_0)$) with the learned reverse transition ($p_{\theta}(x_{t-1}|x_t)$).

So the hierarchy is:

```
[
\boxed{
\text{ELBO}
\rightarrow
\text{contains KL terms}
\rightarrow
\text{one KL compares } q \text{ and } p_{\theta}
\rightarrow
\text{minimizing it trains the reverse model}.
}
]
```

And this is why you should not think that we're arbitrarily choosing two distributions to compare. The comparison arises naturally from the variational lower-bound derivation.

Because once (x_0) is fixed, (x_{t-1}) and (x_t) form a jointly Gaussian pair. Then the standard Gaussian-conditioning theorem applies.

Let's see exactly why.

1. Given (x_0) , (x_{t-1}) is Gaussian

The forward process gives

[

$q(x_{t-1} | x_0)$

```
\mathcal{N}
\left(
x_{t-1};
\sqrt{\bar{\alpha}_{t-1}}x_0,
(1-\bar{\alpha}_{t-1})I
\right).
]
```

So, once (x_0) is fixed,

$$[x_{t-1} \mid x_0]$$

is Gaussian.

2. (x_t) is obtained from (x_{t-1}) by adding Gaussian noise

$$[$$

$$x_t$$

$$\begin{aligned} & \sqrt{\alpha_t} x_{t-1} \\ & + \\ & \sqrt{\beta_t} \epsilon_t, \end{aligned}$$

where

$$[\epsilon_t \sim \mathcal{N}(0, I).]$$

Therefore, given (x_0) , we have the linear-Gaussian model

$$\begin{aligned} & [x_{t-1} \mid x_0 \\ & \rightarrow \\ & x_t \mid x_{t-1}. \end{aligned}$$

Both pieces are Gaussian and the relationship is linear.

Consequently,

$$\boxed{(x_{t-1}, x_t) \mid x_0 \text{ is jointly Gaussian}.}$$

```
}
]
```

3. Now apply Gaussian conditioning

For jointly Gaussian variables, conditioning one on the other gives a Gaussian.

Therefore,

```
[
```

```
\boxed{
```

```
q(x_{t-1} \mid x_t, x_0)
```

```
\mathcal{N}
\left(
x_{t-1};
\tilde{\mu}_t(x_t, x_0),
\tilde{\beta}_t I
\right).
}
]
```

That's the entire reason.

You can also see it directly using Bayes

Start with

```
[
q(x_{t-1} \mid x_t, x_0)
\propto
q(x_t \mid x_{t-1}, x_0)
q(x_{t-1} \mid x_0).
]
```

By the Markov property,

[

$q(x_t \mid x_{t-1}, x_0)$

$q(x_t \mid x_{t-1})$.

]

So

[

$q(x_{t-1} \mid x_t, x_0)$

$\propto$

$\underbrace{q(x_t \mid x_{t-1})}_{\text{Gaussian in } x_{t-1}}$

$\underbrace{q(x_{t-1} \mid x_0)}_{\text{Gaussian in } x_{t-1}}$.

]

Therefore,

[

$\boxed{}$

$\text{Gaussian} \times \text{Gaussian}$

$\propto$

Gaussian .

}

]

Hence the posterior is Gaussian.

The key distinction

[

$\boxed{}$

$q(x_{t-1} \mid x_t, x_0)$

}

]

is Gaussian because (x_0) is fixed, making the diffusion chain a linear-Gaussian system.

But

[

$\boxed{\int q(x_{t-1} \mid x_t, x_0) q(x_0 \mid x_t) dx_0}$

$q(x_{t-1} \mid x_t)$

$\int q(x_{t-1} \mid x_t, x_0) q(x_0 \mid x_t) dx_0$
}
]

is generally a mixture of those Gaussians, because (x_0) is no longer fixed.

So the crucial word is conditioned on (x_0) .

Yes — this is the subtle point, and you're right to challenge it. But there is an important distinction.

$(q(x_{t-1} \mid x_t))$ is not generally Gaussian just because the forward transitions are Gaussian

We know

[
 $q(x_t \mid x_{t-1})$
]

is Gaussian.

And, conditioned on (x_0) ,

[
 $q(x_t \mid x_0)$
]

is Gaussian.

Therefore, conditional on (x_0) , the pair $((x_{t-1}, x_t))$ is jointly Gaussian, and hence

[
 $\boxed{q(x_{t-1} \mid x_t, x_0)}$
}
]

is Gaussian by Gaussian conditioning.

But if we remove (x_0) ,

```
[  
q(x_{t-1} \mid x_t),  
]
```

we have marginalized over (x_0) :

```
[
```

$q(x_{t-1} \mid x_t)$

```
\int q(x_{t-1} \mid x_t, x_0) q(x_0 \mid x_t) dx_0.  
]
```

Here is the key:

```
[  
\boxed{\text{A mixture of Gaussians is not generally Gaussian.}}  
]
```

And $(q(x_0 \mid x_t))$ depends on the data distribution, which is generally highly non-Gaussian.

A simple example

Suppose your data (x_0) consists of two possible values:

```
[  
x_0=-1  
\quad\text{or}\quad  
x_0=+1.  
]
```

After adding Gaussian noise, (x_t) has a mixture-like distribution.

Conditioning on a particular (x_0) , you might have

```
[
```

$$q(x_{t-1} \mid x_t, x_0 = -1)$$

$$\mathcal{N}(\mu_1, \sigma^2)$$

]

and

[

$$q(x_{t-1} \mid x_t, x_0 = +1)$$

$$\mathcal{N}(\mu_2, \sigma^2).$$

]

But without knowing (x_0) ,

[

$$q(x_{t-1} \mid x_t)$$

$$w_1 \mathcal{N}(\mu_1, \sigma^2)$$

+

$$w_2 \mathcal{N}(\mu_2, \sigma^2),$$

]

which is generally not one Gaussian.

So the exact distinction is

[

$\boxed{\phantom{x_{t-1} \mid x_t, x_0}}$

$$q(x_{t-1} \mid x_t, x_0)$$

is Gaussian

]

]

because we're conditioning the jointly Gaussian diffusion variables given (x_0) .

Whereas

```
[
\boxed{
q(x_{t-1} \mid x_t)
\text{ is generally not Gaussian}
}
]
```

because it averages over the unknown (x_0) .

And this is precisely why DDPM introduces the tractable posterior

```
[
q(x_{t-1} \mid x_t, x_0)
]
```

in the ELBO.

So your statement "Gaussian conditioning makes it Gaussian" is correct — but the conditioning includes (x_0) . Removing (x_0) changes the problem into a marginalization/mixture of Gaussians, which does not preserve Gaussianity in general.

I see the exact issue. You're asking:

If (q) is the forward diffusion distribution, why isn't the "true posterior" simply $(q(x_{t-1} \mid x_t))$? Why do DDPM papers use $(q(x_{t-1} \mid x_t, x_0))$?

The answer is: there are two different posteriors being discussed.

1. Mathematically, the marginal posterior is indeed

If you ask:

Given only (x_t) , what is the distribution of (x_{t-1}) ?

then it is

```
[
\boxed{q(x_{t-1} \mid x_t)}.
]
```

That is absolutely a valid posterior.

But it is generally hard to calculate.

Why?

Because to obtain it, you have to integrate over all possible (x_0):

[

$q(x_{t-1} \mid x_t)$

```
\int
q(x_{t-1} \mid x_t, x_0)
q(x_0 \mid x_t)
, dx_0.
]
```

So ($q(x_{t-1} \mid x_t)$) is a marginalized posterior.

2. The tractable posterior is

During training, we actually know (x_0).

Therefore we can work with

```
[
\boxed{
q(x_{t-1} \mid x_t, x_0)
}
]
```

instead.

This is analytically tractable and Gaussian.

So:

```
[
\boxed{
q(x_{t-1} \mid x_t, x_0)
\text{ is NOT the same as }
q(x_{t-1} \mid x_t)
}
```

```
}  
]
```

in general.

3. Why does the model learn $(p_{\theta}(x_{t-1}|x_t))$?

Because during generation, (x_0) is unavailable.

We need:

```
[  
  \boxed{  
    p_{\theta}(x_{t-1}|x_t)  
  }  
]
```

which only takes (x_t) as input.

So the situation is:

Distribution	Meaning	Available?
$(q(x_{t-1} x_t, x_0))$	Training-time tractable posterior	Yes, because (x_0) is known
$(q(x_{t-1} x_t))$	True posterior given only (x_t)	Hard to calculate
$(p_{\theta}(x_{t-1} x_t))$	Learned reverse model	What we want

4. The really interesting part

The model is ultimately trying to approximate the reverse distribution

$$[q(x_{t-1} | x_t).]$$

But instead of directly computing this difficult distribution, the ELBO gives us a tractable training objective involving

$$[q(x_{t-1} | x_t, x_0).]$$

That's why the extra (x_0) appears.

So your intuition is correct:

[

$\boxed{\text{True posterior given only } x_t}$

$$[q(x_{t-1} | x_t)]$$

but DDPM uses

$$[\boxed{q(x_{t-1} | x_t, x_0)}]$$

because that conditional posterior is tractable during training, and the ELBO lets us use it to train the model that will operate without (x_0) .

Exactly. That statement is correct.

$$[\boxed{q(x_t | x_{t-1})} \text{ Gaussian}]$$

does not by itself imply

$$[\boxed{q(x_{t-1} | x_t) \text{ Gaussian}.}]$$

Why?

Bayes' rule gives

[

$q(x_{t-1} \mid x_t)$

$$\frac{q(x_t \mid x_{t-1})q(x_{t-1})}{q(x_t)}.$$

]

We know

[
 $q(x_t \mid x_{t-1})$
]

is Gaussian as a function of (x_t) .

But to determine the reverse distribution, we also need the prior

[
 $q(x_{t-1})$.
]

If $(q(x_{t-1}))$ is non-Gaussian, the resulting posterior generally isn't Gaussian.

In DDPM, what changes?

We condition on (x_0) :

[
 $q(x_{t-1} \mid x_t, x_0)$.
]

Now we know

[

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)$$

$\mathcal{N}(\cdot)$

]

and

[

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

$\mathcal{N}(\cdot)$.

]

Therefore,

[

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$$

$\propto$

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_0),$$

]

which is

[

$\text{Gaussian} \times \text{Gaussian}$

$\propto$

Gaussian .

]

Thus:

[

$\boxed{}$

$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$ is Gaussian.


```
}  
]
```

The clean distinction

```
[  
\boxed{  
q(x_t|x_{t-1})\text{ Gaussian}  
\not\rightarrow  
q(x_{t-1}|x_t)\text{ Gaussian}  
}  
]
```

but

```
[  
\boxed{  
q(x_{t-1}|x_0)\text{ Gaussian}  
\quad+\quad  
q(x_t|x_{t-1})\text{ Gaussian}  
\rightarrow  
q(x_{t-1}|x_t, x_0)\text{ Gaussian}.  
}  
]
```

This is exactly why (x_0) appears in the DDPM posterior used in the ELBO.

Absolutely. This section is the most important mathematical bridge in the original DDPM paper: it explains why the authors stop thinking of the reverse model as merely predicting a Gaussian mean and instead parameterize it so that the neural network predicts the noise (ϵ).

The whole section can be understood as:

```
[  
\boxed{  
\text{ELBO}  
\rightarrow  
\text{Gaussian KL}  
\rightarrow  
\text{predict reverse mean}  
\rightarrow  
\text{reparameterize mean}  
\rightarrow  
\text{predict noise}  
}
```

```

\rightarrow
\text{MSE noise loss}
}
]

```

I'll go through it carefully.

1. Where we are starting

From the previous section, the diffusion model has two processes.

Forward process

[

$$q(x_{1:T}|x_0)$$

```

\prod_{t=1}^T q(x_t|x_{t-1})
]

```

with

[

$$q(x_t|x_{t-1})$$

```

\mathcal{N}
\left(
x_t;
\sqrt{\alpha_t}x_{t-1},
\beta_t I
\right)
]

```

where

$$\begin{bmatrix} \alpha_t = 1 - \beta_t \\ \end{bmatrix}$$

and

$$\begin{bmatrix} \bar{\alpha}_t = \prod_{s=1}^t \alpha_s. \\ \end{bmatrix}$$

The important closed form is

$$\begin{bmatrix}$$

$\boxed{\{$

x_t

$$\begin{bmatrix} \sqrt{\bar{\alpha}_t} x_0 \\ + \\ \sqrt{1 - \bar{\alpha}_t} \epsilon \\ \} \\ \end{bmatrix}$$

with

$$\begin{bmatrix} \epsilon \sim \mathcal{N}(0, I). \\ \end{bmatrix}$$

Reverse process

The model learns

[

$$p_{\theta}(x_{t-1}|x_t)$$

```
\mathcal{N}
\left(
x_{t-1};
\mu_{\theta}(x_t,t),
\Sigma_{\theta}(x_t,t)
\right).
]
```

So the question now is:

What should the neural network predict?

There are several possibilities.

It could predict:

1. the reverse-process mean (μ_{θ}),
2. the original image (x_0),
3. the noise (ϵ).

The authors discover that predicting (ϵ) works particularly well.

2. Why does the paper say there are many degrees of freedom?

The paper begins by saying that diffusion models may look restrictive, but there are actually many choices.

For example:

Forward process

You must choose

[
 $\beta_1, \dots, \beta_T$.
]

Reverse process

You must choose

[
 $\Sigma_\theta(x_t, t)$.
]

Neural network

You must choose its architecture.

For example:

[
 U-Net .
]

Parameterization

You must decide what the neural network predicts.

Possibilities include:

[
 μ_θ
]

or

[
 x_0
]

or

[
 ϵ .
]

The paper's main contribution in this section is showing that the noise prediction parameterization has particularly nice mathematical properties.

3. Section 3.1: Forward process and (L_T)

They say:

We fix (β_t) to constants.

This means the forward process is not learned.

We choose:

[
 $\beta_1, \beta_2, \dots, \beta_T$
]

before training.

Therefore:

[
 $q(x_{1:T}|x_0)$
]

has no trainable parameters.

Why does this make (L_T) constant?

Recall:

[

L_T

D_{KL}
 $\left(q(x_T|x_0) \right.$
|

$p(x_T)$
 $\right).$
 $]$

Since:

- q is fixed,
- $p(x_T) = \mathcal{N}(0, I)$ is fixed,

there is no θ involved.

Therefore:

$$\boxed{\nabla_{\theta} L_T = 0.}$$

So during neural-network training, L_T contributes nothing to the gradient.

Hence:

$$\boxed{\text{Ignore } L_T \text{ during training.}}$$

This is a very useful simplification.

4. Section 3.2: Reverse process

Now we focus on

$$L_1, \dots, L_{T-1}.$$

The reverse process is

$$[$$

$p_{\theta}(x_{t-1}|x_t)$

```
\mathcal{N}
(
x_{t-1};
\mu_{\theta}(x_t,t),
\Sigma_{\theta}(x_t,t)
).
```

The authors first decide what to do with the covariance.

5. They fix the covariance

They choose:

```
[
\boxed{
\Sigma_{\theta}(x_t,t)=\sigma_t^2I.
}
]
```

Notice something important.

The covariance is now:

- isotropic,
- independent of (x_t) ,
- independent of (θ) .

It only depends on timestep (t) .

So the network only needs to learn the mean.

6. Two choices for (σ_t^2)

The authors experiment with two choices.

Choice 1

```
[  
\boxed{  
\sigma_t^2=\beta_t  
}  
]
```

Choice 2

```
[  
\boxed{  
\sigma_t^2=\tilde{\beta}_t  
}  
]
```

where

[

$\tilde{\beta}_t$

```
\frac{1-\bar{\alpha}_{t-1}}  
{1-\bar{\alpha}_t}  
\beta_t.  
]
```

Both work reasonably similarly.

7. Why are these two values special?

Remember:

[

$$q(x_{t-1}|x_t, x_0)$$

```
\mathcal{N}
(
x_{t-1};
\tilde{\mu}_t,
\tilde{\beta}_t)
]
```

So the actual forward posterior has variance:

```
[
\tilde{\beta}_t.
]
```

That makes it natural to use

```
[
\sigma_t^2=\tilde{\beta}_t.
]
```

But the authors also try:

```
[
\sigma_t^2=\beta_t.
]
```

They find both work.

8. Now look at (L_{t-1})

Recall:

```
[
```

L_{t-1}

$$D_{\text{KL}} \left(q(x_{t-1} | x_t, x_0) \parallel p_{\theta}(x_{t-1} | x_t) \right).$$

We have:

$$[$$

$$q(x_{t-1} | x_t, x_0)$$

$$\mathcal{N} \left(\tilde{\mu}_t, \tilde{\beta}_t I \right)$$

and

$$[$$

$$p_{\theta}(x_{t-1} | x_t)$$

$$\mathcal{N} \left(\mu_{\theta}, \sigma_t^2 I \right).$$

The KL divergence between two Gaussians has a closed form.

Since (σ_t^2) is fixed, all terms that don't depend on (θ) can be absorbed into a constant (C) .

The result is:

[

$\boxed{}$

L_{t-1}

$\mathbb{E}_q$

$\left[$

$\frac{1}{2\sigma_t^2}$

$\left|$

$\tilde{\mu}_t(x_t, x_0)$

$\mu_{\theta}(x_t, t)$

$\right]^2$

$\right]$

$-C.$

$\}$

$]$

The paper writes this as Equation (8).

9. This is a major result

Equation (8) tells us:

To minimize the KL divergence, the neural network should make its predicted mean close to the true posterior mean.

In other words:

```
[
\boxed{
\mu_{\theta}(x_t,t)
\approx
\tilde{\mu}_t(x_t,x_0).
}
]
```

So the most obvious strategy would be:

```
[
\boxed{
\text{Train the network to predict }\tilde{\mu}_t.
}
]
```

That is what the paper calls $(\tilde{\mu})$ -prediction.

10. But there is a problem

The true posterior mean is

```
[
\tilde{\mu}_t(x_t,x_0).
]
```

It depends on:

```
[
x_0.
]
```

But during generation we don't know (x_0) !

We only have:

```
[
x_t.
]
```

So the network cannot literally receive (x_0) .

This means we need to rewrite

$$[\tilde{\mu}_t(x_t, x_0)]$$

in a form involving something the network can predict from (x_t) .

And this is where the magic happens.

11. Remember the direct noising equation

We have:

$$[$$

$$x_t$$

$$\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon.$$

$$]$$

Solve this for (x_0) .

Subtract the noise:

$$[$$

$$x_t - \sqrt{1-\bar{\alpha}_t}\epsilon$$

$$\sqrt{\bar{\alpha}_t}x_0.$$

$$]$$

Therefore:

[

$\boxed{\frac{x_t - \sqrt{1 - \alpha_t} \epsilon}{\sqrt{\alpha_t}}}$

x_0

$$\frac{x_t - \sqrt{1 - \alpha_t} \epsilon}{\sqrt{\alpha_t}}$$

This is crucial.

It says:

If I know (x_t) and the noise (ϵ) , I can recover (x_0) .

12. But the network doesn't know (ϵ) either

Exactly.

During training, we know (ϵ) , because we generated it.

But during generation, we don't know the noise that originally produced (x_t) .

So we train a neural network:

$$\epsilon_{\theta}(x_t, t)$$

to predict it.

That is the central idea.

13. Substitute (x_0) into the posterior mean

We previously had:

[

$\tilde{\mu}_t(x_t, x_0)$

```
\frac{
\sqrt{\bar{\alpha}_{t-1}}\beta_t
}{
1-\bar{\alpha}_t
}x_0
+
\frac{
\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})
}{
1-\bar{\alpha}_t
}x_t.
]
```

Now substitute:

[

x_0

$$\frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon}{\sqrt{\bar{\alpha}_t}}.$$

After simplifying, the posterior mean becomes:

$$[$$

$$\boxed{\mu_t}$$

$$\tilde{\mu}_t$$

$$\frac{1}{\sqrt{\alpha_t}}$$

$$\left($$

$$x_t$$

$$\frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t} \epsilon} \right).$$

This is the key equation behind the DDPM noise-prediction parameterization.

14. Read that equation carefully

We now have:

[

$\tilde{\mu}_t$

$\frac{1}{\sqrt{\alpha_t}}$

$\left($

x_t

$\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}$
 ϵ
 $\right).$
]

Everything is known except:

[
 ϵ .
]

So instead of asking the neural network to directly predict

[
 $\tilde{\mu}_t$,
]

we can ask it to predict:

[
 ϵ .
]

Then construct the mean ourselves.

15. This gives Equation (11)

Define:

[
 $\epsilon_\theta(x_t, t)$
]

as a neural network predicting the noise.

Then:

[

$\boxed{\epsilon_\theta(x_t, t)}$

$\mu_\theta(x_t, t)$

$\frac{1}{\sqrt{\alpha_t}}$

$\left($

x_t

$\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t)$
 $\right)$
]

This is Equation (11).

This is one of the most important equations in the DDPM paper.

16. What does the network actually output?

The network takes:

```
[  
( $x_t$ , t)  
]
```

and outputs:

```
[  
\boxed{  
\epsilon_\theta(x_t, t).  
}  
]
```

It does not directly output:

```
[  
 $x_{t-1}$ .  
]
```

It does not necessarily directly output:

```
[  
 $x_0$ .  
]
```

It predicts:

```
[  
\boxed{\text{the noise contained in } x_t.}  
]
```

Then the mathematical formula converts that noise prediction into the reverse-process mean.

17. Why is noise prediction so clever?

Because the forward process is:

```
[
x_t=
\underbrace{\sqrt{\bar{\alpha}_t}x_0}_{\text{signal}}
+
\underbrace{\sqrt{1-\bar{\alpha}_t}\epsilon}_{\text{noise}}.
]
```

The network receives (x_t) .

Its task is essentially:

"Tell me which part of this noisy image is noise."

Once it predicts the noise:

```
[
\epsilon_\theta,
]
```

we can estimate the clean signal.

18. The training algorithm now becomes incredibly simple

This is Algorithm 1.

Let's go line by line.

Step 1

```
[
x_0 \sim q(x_0).
]
```

Take a real training image.

For example:

```
[  
x_0=\text{cat image}.  
]
```

Step 2

```
[  
t\sim\text{Uniform}(\{1,\ldots,T\}).  
]
```

Randomly choose a timestep.

For example:

```
[  
t=700.  
]
```

Step 3

```
[  
\epsilon\sim\mathcal{N}(0,I).  
]
```

Generate random Gaussian noise.

Step 4

Construct:

```
[
```

x_t

$$\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon.$$

Now you have a noisy version of the image.

Step 5

Feed:

$$(x_t, t)$$

to the neural network.

It predicts:

$$\epsilon_{\theta}(x_t, t).$$

Step 6

Compare:

$$\epsilon_{\theta}(x_t, t)$$

with the actual known noise:

$$\epsilon.$$

The loss is:

$$\boxed{\| \epsilon_{\theta}(x_t, t) - \epsilon \|^2}$$

```
\left|
\epsilon - \epsilon_{\theta}(x_t, t)
\right|^2.
}
]
```

That's it.

19. This is why the training loop looks so simple

The entire algorithm is basically:

```
[
\boxed{
\text{clean image}
\rightarrow
\text{add known noise}
\rightarrow
\text{predict noise}
\rightarrow
\text{compare with actual noise}.
}
]
```

It looks like ordinary supervised learning.

The target is:

```
[
\epsilon.
]
```

The prediction is:

```
[
\epsilon_{\theta}(x_t, t).
]
```

20. But where did the ELBO go?

This is a very important question.

The paper didn't just arbitrarily invent:

$$\left[\begin{array}{l} \\ |\epsilon - \epsilon_{\theta}|^2. \\ \end{array} \right]$$

It started from:

[

$$L_{t-1}$$

$$\left[\begin{array}{l} D_{\text{KL}}(q|p_{\theta}). \\ \end{array} \right]$$

Then:

$$\left[\begin{array}{l} \\ \text{Gaussian KL} \\ \end{array} \right]$$

gave:

$$\left[\begin{array}{l} \\ |\tilde{\mu}_t - \mu_{\theta}|^2. \\ \end{array} \right]$$

Then they reparameterized:

$$\left[\begin{array}{l} \\ \tilde{\mu}_t \\ \end{array} \right]$$

in terms of

$$\left[\begin{array}{l} \\ \epsilon. \\ \end{array} \right]$$

Then they parameterized:

$$[\mu_{\theta}]$$

in terms of

$$[\epsilon_{\theta}.$$

And finally the KL term becomes a weighted noise-prediction MSE.

So:

$$\boxed{\begin{aligned} D_{\text{KL}} &\rightarrow |\tilde{\mu} - \mu_{\theta}|^2 \\ &\rightarrow |\epsilon - \epsilon_{\theta}|^2. \end{aligned}}$$

That is the mathematical story.

21. Equation (12)

After substituting the noise-prediction parameterization into Equation (10), the authors obtain:

$$[$$

$$\boxed{\begin{aligned} & \end{aligned}}$$

$L_{t-1}-C$

$$\mathbb{E}_{x_0, \epsilon} \left[\frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \left| \epsilon - \epsilon_{\theta(x_t, t)} \right|^2 \right]$$

This is extremely important.

It says:

The variational objective is equivalent to a weighted noise-prediction MSE.

The weight is:

$$\boxed{w_t = \frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)}}$$

So the exact variational objective is not simply unweighted MSE.

It is:

$$\boxed{\text{weighted MSE.}}$$

22. Why does the paper talk about denoising score matching?

Now we reach the deeper theoretical connection.

The score of a probability distribution is:

$$\boxed{\nabla_x \log p(x)}.$$

Suppose you have a noisy image (x_t).

The score tells you how the probability density changes around that point.

Intuitively, it points toward regions of higher probability.

For noisy data, this information can tell us how to move from:

$$\text{noise} \rightarrow \text{data}.$$

23. Gaussian score

Suppose:

$$[$$
$$x_t$$

$$\sqrt{\bar{\alpha}_t} x_0 +$$

$\sqrt{1-\bar{\alpha}_t}\epsilon.$
]

Conditioned on (x_0) ,

[

$q(x_t|x_0)$

$\mathcal{N}$
(
 x_t ;
 $\sqrt{\bar{\alpha}_t}x_0$,
 $(1-\bar{\alpha}_t)I$
).
]

The score of this Gaussian with respect to (x_t) is:

[

$\nabla_{x_t} \log q(x_t|x_0)$

$-\frac{x_t - \sqrt{\bar{\alpha}_t}x_0}{1-\bar{\alpha}_t}$.
]

But:

[

$x_t - \sqrt{\bar{\alpha}_t}x_0$

$\sqrt{1-\bar{\alpha}_t}\epsilon.$
]

Therefore:

[

$\boxed{}$

$\nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t | \mathbf{x}_0)$

$-\frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}}.$
}
]

This is a beautiful result.

24. Noise prediction is therefore related to score prediction

If:

[

$\mathbf{s}(\mathbf{x}_t, t)$

$\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t),$
]

then approximately:

```
[
\boxed{
s_{\theta}(x_t,t)
\propto
-\frac{
\epsilon_{\theta}(x_t,t)
}{
\sqrt{1-\bar{\alpha}_t}
}
}
]
```

More specifically, under the appropriate marginal interpretation:

```
[
\boxed{
\epsilon_{\theta}(x_t,t)
\approx
-\sqrt{1-\bar{\alpha}_t},
\nabla_{\{x_t\}} \log q_t(x_t).
}
]
```

So predicting noise is essentially another way of predicting the score.

That is why the paper says the objective resembles denoising score matching.

25. What is denoising score matching intuitively?

Suppose you take clean data:

```
[
x_0.
]
```

Add noise:

```
[
x_t.
]
```

Then train a network to determine how the noisy sample should move toward the clean-data distribution.

You are effectively teaching the model:

```
[
\boxed{
\text{"Given this noisy point, which direction leads toward more likely data?"}
}
]
```

Diffusion does this for many noise levels:

```
[
t=1,2,\ldots,T.
]
```

So the network learns denoising behavior at multiple scales.

26. Why "multiple noise scales"?

Because t controls noise level.

For small t :

```
[
x_t \approx x_0.
]
```

For large t :

```
[
x_t \approx \epsilon.
]
```

Therefore the network learns:

Small noise

```
[
\text{slightly corrupted image}
\rightarrow
```


clean image .
]

Medium noise

[
 $\text{moderately corrupted image}$
 $\rightarrow$
 structured image .
]

Large noise

[
 almost pure noise
 $\rightarrow$
 data structure .
]

Thus one network learns denoising across many noise levels.

27. Why does the paper mention Langevin dynamics?

The reverse sampling equation is:

[

x_{t-1}

$\frac{1}{\sqrt{\alpha_t}}$

$\left($

$\mathbf{x}_t$

$$\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) + \sigma_t z$$

where

$$z \sim \mathcal{N}(0, I)$$

There are two components:

Directional movement

$$-\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t)$$

Random noise

$$\sigma_t z$$

This resembles Langevin dynamics, which roughly has the form:

$$[$$

$\mathbf{x}'$

$$\eta \nabla_{\mathbf{x}} \log p(\mathbf{x}) +$$

$\sqrt{2\eta}z$.
]

That is:

[
\boxed{
\text{move toward high-density regions}
+
\text{add random noise}.
}
]

Diffusion sampling has a similar structure.

28. Why is this significant?

The authors are saying something deeper than:

"Our network predicts noise."

They are saying:

"Our diffusion model can also be interpreted as learning the score of the data distribution at different noise levels, and its sampling procedure resembles Langevin dynamics."

So there are two interpretations:

Probabilistic latent-variable interpretation

[
\boxed{
\text{variational inference}
}
]

Score-based interpretation

[
\boxed{
\text{denoising score matching + Langevin-like sampling}
}

}
]

And surprisingly, they describe essentially the same model.

29. The paper's major conceptual result

This sentence is the heart of Section 3.2:

optimizing an objective resembling denoising score matching is equivalent to using variational inference to fit the finite-time marginal of a sampling chain resembling Langevin dynamics.

In simpler language:

[

\boxed{

\text{DDPM}

\text{variational inference view}

\text{score matching view}

}
]

at least under the particular parameterization and formulation being discussed.

That's a major conceptual connection.

30. Why not predict (x_0) ?

The paper says there is another possibility:

[
 x_0 \text{-prediction}.
]

Because:

[

x_t

$\sqrt{\bar{\alpha}_t}x_0 +$
 $\sqrt{1-\bar{\alpha}_t}\epsilon,$
]

you could train:

[
 $x_{\theta(x_t, t)} \approx x_0.$
]

Then use the predicted (x_0) to calculate the reverse mean.

Mathematically this is possible.

But the authors found that it produced worse sample quality in their early experiments.

So they choose:

[
 $\boxed{\epsilon \text{-prediction}.}$
]

31. Three possible parameterizations

This is worth remembering.

A. Predict posterior mean

```
[  
  \boxed{  
    \mu_{\theta}(x_t,t)  
    \approx  
    \tilde{\mu}_t(x_t,x_0)  
  }  
]
```

B. Predict clean data

```
[  
  \boxed{  
    x_{\theta}(x_t,t) \approx x_0  
  }  
]
```

C. Predict noise

```
[  
  \boxed{  
    \epsilon_{\theta}(x_t,t) \approx \epsilon  
  }  
]
```

The original DDPM paper finds C particularly effective.

32. Why predicting noise is mathematically convenient

Because the forward process is already written as:

```
[
```

x_t

$\sqrt{\bar{\alpha}_t}x_0$
+
 $\sqrt{1-\bar{\alpha}_t}\epsilon.$
]

So (ϵ) is explicitly available during training.

You literally sampled it yourself.

Thus the target is known exactly:

[
 $\epsilon.$
]

No separate inference procedure is needed to create the target.

33. Section 3.3: Why scale image data to $[-1,1]$?

The paper says image pixels are originally:

[
0,1,...,255.
]

They transform them to:

[
[-1,1].
]

For example:

[
0 $\rightarrow$ -1
]

and

```
[
255\rightarrow 1.
]
```

A standard transformation is:

```
[
```

x_{scaled}

```
2\frac{x}{255}-1.
]
```

34. Why do this?

The reverse process starts from:

```
[
x_T \sim \mathcal{N}(0, I).
]
```

A standard Gaussian is centered around:

```
[
0.
]
```

So it is convenient for the model to operate in a data space also centered around zero.

Instead of:

```
[
[0, 255],
]
```

we use:


```
[  
[-1,1].  
]
```

This gives more consistent numerical scales.

35. Why is (L_0) special?

The model ultimately needs to produce a discrete image.

Pixel values are integers:

```
[  
0,1,\ldots,255.  
]
```

But the Gaussian reverse process produces continuous values.

For example, the network might generate:

```
[  
127.43.  
]
```

A likelihood model needs to properly account for the fact that the actual observed value is a discrete integer.

Therefore, the paper uses a discretized Gaussian decoder for the final step.

36. Understanding the final decoder

They define:

```
[
```

$p_{\theta}(x_0|x_1)$

$\prod_{i=1}^D \int_{\delta_-(x_0^i)}^{\delta_+(x_0^i)} \mathcal{N}(x; \mu_{\theta^i}(x_1, 1), \sigma_1^2) dx.$

The idea is simple.

Suppose the observed pixel is:

$[x_0^i = 127.]$

Instead of asking:

What is the probability density exactly at 127?

we ask:

What probability mass falls into the interval corresponding to pixel value 127?

Approximately:

$[126.5, 127.5].$

Therefore:

[

$P(x_0^i = 127)$

```
\int_{126.5}^{127.5}
p(x),dx.
]
```

This converts the continuous Gaussian into a probability distribution over discrete pixel values.

37. Why is this important for likelihood?

If you simply evaluate:

```
[
\mathcal{N}(127;\mu,\sigma^2)
]
```

you are evaluating a density, not the probability of the discrete pixel value.

The discretized decoder integrates over the appropriate bin.

This makes the model's likelihood mathematically appropriate for discrete image data.

38. Why does the paper say "lossless codelength"?

Because likelihood can be interpreted in terms of compression.

If the model assigns probability:

```
[
p(x)
]
```

to data (x), its ideal code length is roughly:

```
[
-\log_2 p(x).
]
```

So a good likelihood model corresponds to an efficient description of the data.

The paper later investigates where the bits are being spent.

39. Section 3.4: Simplified training objective

Now comes one of the most important practical decisions.

The exact variational objective gives:

$$\left[\begin{array}{l} L_{t-1} \\ \propto \\ w_t \\ \left| \epsilon - \epsilon_{\theta(x_t, t)} \right|^2 \end{array} \right]$$

where

$$[$$
$$w_t$$

$$\frac{\beta_t^2}{2\sigma_t^2\alpha_t(1-\bar{\alpha}_t)}.$$

But the authors say:

We found it beneficial to sample quality to train using a simpler objective.

They remove the weighting.

Thus:

[

\boxed{

L_{\text{simple}}

\mathbb{E}_{t,x_0,\epsilon}\left[|\epsilon-\epsilon_{\theta}(x_t,t)|^2\right].

This is Equation (14).

40. This is the famous DDPM loss

When people say:

"The DDPM loss is MSE between true noise and predicted noise."

they are usually referring to:

[

\boxed{

L_{\text{simple}}

$$\mathbb{E} \left[\epsilon_{\theta(x_t, t)}^2 \right]$$

The practical training algorithm is therefore extremely simple.

41. But is this exactly the ELBO?

No.

This distinction is very important.

The exact variational objective contains timestep-dependent weighting:

$$\sum_t w_t \epsilon_{\theta}^2.$$

The simplified objective removes those weights:

$$\sum_t \epsilon_{\theta}^2.$$

Therefore:

$$\boxed{L_{\text{simple}} \neq L_{\text{VLB}}}$$

in a strict mathematical sense.

Instead:

```
[
\boxed{
L_{\text{simple}}
\text{ is a simplified/weighted variational objective.}
}
]
```

The authors empirically find that it produces better samples.

42. Why removing the weights helps?

This is a subtle but very interesting point.

Different timesteps correspond to different noise levels.

For small (t):

```
[
x_t \approx x_0.
]
```

The image is only slightly noisy.

The task is:

```
[
\text{easy denoising}.
]
```

For large (t):

```
[
x_t \approx \epsilon.
]
```

The task is much harder:

```
[
\text{recover structure from heavy noise}.
]
```

The exact variational weighting can cause the model to spend disproportionate attention on some noise levels.

The authors find that removing the weighting allows the network to focus more on difficult denoising tasks.

43. What does "down-weight small (t)" mean?

The paper says:

our diffusion process setup ... causes the simplified objective to down-weight loss terms corresponding to small (t).

Think of:

(t=1)

Almost-clean image:

[
 $x_1 \approx x_0$.
]

Predicting the noise is relatively easy.

(t=T)

Almost pure noise:

[
 $x_T \approx \epsilon$.
]

The network has to understand much more global structure.

So the authors don't want the model to spend too much capacity perfecting trivial low-noise denoising.

Instead they want it to become good at:

[
 $\boxed{\text{harder high-noise denoising}}$.
]


```
}  
]
```

44. Why does this improve sample quality?

The ultimate goal during generation is to start from:

```
[  
x_T \sim \mathcal{N}(0, I).  
]
```

Therefore, early reverse steps are operating in extremely noisy regions.

The model needs to know how to transform:

```
[  
\text{almost pure noise}  
\rightarrow  
\text{coarse structure}.  
]
```

If the network is excellent only at tiny amounts of noise, that isn't enough.

So emphasizing difficult noise levels can improve generation.

45. Algorithm 2: Sampling

Now let's understand the generation algorithm.

Step 1

Sample:

```
[  
  \boxed{  
    x_T \sim \mathcal{N}(0, I).  
  }  
]
```

This is pure Gaussian noise.

Step 2

For:

```
[  
  t=T, T-1, \ldots, 1  
]
```

repeat.

Step 3

If:

```
[  
  t > 1,  
]
```

sample:

```
[  
  z \sim \mathcal{N}(0, I).  
]
```

But if:

```
[  
  t = 1,  
]
```

set:

```
[
z=0.
]
```

Why?

Because at the final step we want the final generated image without adding another random perturbation.

46. Compute the reverse step

The paper uses:

```
[
```

```
\boxed{
```

```
x_{t-1}
```

```
\frac{1}{\sqrt{\alpha_t}}
```

```
\left(
```

```
x_t
```

```
\frac{\beta_t}{
\sqrt{1-\bar{\alpha}_t}}
\epsilon_{\theta}(x_t,t)
\right)
+
\sigma_t z.
}
```

Break this into two parts.

Part 1

[

$\frac{1}{\sqrt{\alpha_t}}$

$\left($

x_t

$\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}$
 ϵ_θ
 $\right)$
]

is the predicted mean:

[
 $\mu_\theta(x_t, t).$
]

Part 2

[
 σ_t
]

adds stochasticity.

Thus:

[

$\boxed{}$

$$\mathbf{x}_{t-1}$$

predicted mean
 $+$
 $\text{random Gaussian variation}$.
 $\}$
 $\]$

47. Why add random noise during reverse sampling?

Because the reverse distribution is Gaussian:

[

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

$N(\mu_{\theta}, \sigma_t^2)$.
 $\]$

Sampling from this Gaussian means:

[

$$\mathbf{x}_{t-1}$$

μ_{θ}
 $+$
 $\sigma_t z$.
 $\]$

If you always used only:

[
 $x_{t-1} = \mu_{\theta}$,
]

you would not actually sample from the learned distribution.

The random term lets the model generate diverse samples.

48. A subtle but important point

The randomness in generation is not just the initial noise.

There is:

[
 $x_T \sim \mathcal{N}(0, I)$
]

and then additional:

[
 $z_t \sim \mathcal{N}(0, I)$
]

at intermediate reverse steps.

So generation is stochastic at multiple stages.

That is why repeated sampling gives different images.

49. The complete sampling process visually

Imagine:

[

x_T

$\text{\text{random static}}$.
]

The network predicts:

[
 $\epsilon_{\theta}(x_T, T)$.
]

We remove some noise:

[
 x_{T-1} .
]

Then:

[
 x_{T-1}
 $\rightarrow$
 x_{T-2} .
]

Eventually:

[
 $\cdots$
 $\rightarrow$
 x_2
 $\rightarrow$
 x_1
 $\rightarrow$
 x_0 .
]

So:

[
 $\boxed{\text{\text{random noise}}}$
 $\rightarrow$
 $\text{\text{coarse structure}}$
 $\rightarrow$

```

\text{recognizable structure}
\rightarrow
\text{fine details}
}
]

```

50. Why does the reverse process need (t) ?

Suppose the network receives the same-looking noisy input at different noise levels.

It needs to know whether it should remove:

- a tiny amount of noise,
- a moderate amount,
- a huge amount.

Therefore:

```

[
\epsilon_{\theta}(x_t, t)
]

```

depends on (t) .

The timestep embedding tells the network the current noise level.

51. The connection to denoising autoencoders

Now let's address the title:

Diffusion models and denoising autoencoders

A denoising autoencoder does something like:


```
[
x_0
\rightarrow
\text{corrupt}(x_0)
\rightarrow
\tilde{x}
]
```

and trains a neural network to reconstruct:

```
[
\tilde{x}
\rightarrow
x_0.
]
```

For example:

```
[
\text{clean image}
\rightarrow
\text{noisy image}
\rightarrow
\text{denoised image}.
]
```

Diffusion does something related, but much more systematically.

52. Ordinary denoising autoencoder

A DAE might choose one corruption process:

```
[
\tilde{x} = x + \text{noise}.
]
```

Then train:

```
[
f_{\theta}(\tilde{x}) \approx x.
]
```

So:

```
[  
  \boxed{  
    \text{one/few noise levels}  
    \rightarrow  
    \text{denoise}.  
  }  
]
```

53. Diffusion model

A diffusion model creates many noise levels:

```
[  
  x_1, x_2, \ldots, x_T.  
]
```

The network learns:

```
[  
  \epsilon_{\theta}(x_t, t).  
]
```

Thus:

```
[  
  \boxed{  
    \text{many noise levels}  
    \rightarrow  
    \text{denoise at every level}.  
  }  
]
```

The timestep (t) tells the network which corruption level it is dealing with.

54. But there is a deeper difference

A denoising autoencoder typically says:

```
[  
\text{noisy image}\rightarrow\text{clean image}.  
]
```

A diffusion model learns a sequence of local denoising transitions:

```
[  
x_T  
\rightarrow  
x_{T-1}  
\rightarrow  
x_{T-2}  
\rightarrow  
\cdots  
\rightarrow  
x_0.  
]
```

So instead of asking the model to perform one huge transformation:

```
[  
\text{noise}\rightarrow\text{image},  
]
```

we ask it to perform many small transformations:

```
[  
\text{noise}  
\rightarrow  
\text{slightly less noise}  
\rightarrow  
\cdots  
\rightarrow  
\text{image}.  
]
```

This makes the generative process tractable.

55. The deepest connection

A denoising autoencoder learns:

```
[
\boxed{
\text{how to denoise corrupted data}.
}
]
```

Diffusion models learn:

```
[
\boxed{
\text{how to denoise corrupted data at many noise levels}.
}
]
```

And because the model learns this denoising behavior over a carefully designed sequence of noise levels, that denoising process can be iterated backward to generate new data.

That's the key leap from ordinary denoising to generation.

56. One very useful mental model

Think of a diffusion model as a denoising machine with a knob.

The knob is t .

At:

```
[
t=1
]
```

the image is almost clean.

At:

```
[
t=500
]
```

it is moderately noisy.

At:

```
[  
t=1000  
]
```

it is almost pure noise.

The network learns:

```
[
```

```
\boxed{
```

```
\text{denoising function}
```

```
f_\theta(x_t,t).  
}  
]
```

So the same neural network knows how to denoise at every level.

57. Why can a denoising model generate completely new images?

This is the most surprising part.

Suppose you start from a random:

```
[  
x_T \sim \mathcal{N}(0, I).  
]
```

The network has learned what valid data looks like at different noise levels.

So it gradually moves the random point toward regions corresponding to the learned data distribution.

Thus:

```
[
\text{random Gaussian point}
\rightarrow
\text{high-probability data region}.
]
```

Repeated many times:

```
[
\boxed{
\text{Gaussian noise}
\rightarrow
\text{sample from data distribution}.
}
]
```

58. Now connect everything mathematically

The complete chain is:

Forward process

```
[
x_0
\rightarrow
x_t
]
```

where

[

x_t

$$\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon.$$

Network

$$\begin{aligned} & (x_t, t) \\ & \rightarrow \epsilon_\theta(x_t, t). \end{aligned}$$

Reverse mean

[

$$\mu_\theta(x_t, t)$$

$$\begin{aligned} & \frac{1}{\sqrt{\alpha_t}} \\ & \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right). \end{aligned}$$

Reverse sampling

[

$$x_{t-1}$$

$$\mu_\theta(x_t, t) + \sigma_{t,z}.$$

Therefore:

```
[
\boxed{
x_T
\rightarrow
x_{T-1}
\rightarrow
\cdots
\rightarrow
x_0.
}
]
```

59. The mathematical derivation in one chain

This is probably the most important derivation to memorize conceptually.

Start with:

[

$L_{\{t-1\}}$

```
D_{KL}
\left(
q(x_{t-1}|x_t,x_0)
|
p_{\theta}(x_{t-1}|x_t)
\right).
]
```

Both are Gaussian.

Therefore:

[

L_{t-1}

$$\frac{1}{2\sigma_t^2} \mathbb{E} \left[|\tilde{\mu}_t - \mu_{\theta}|^2 \right] + C.$$

Then use:

$$x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon.$$

Therefore:

$$x_0 = \frac{x_t - \sqrt{1 - \alpha_t} \epsilon}{\sqrt{\alpha_t}}.$$

Substitute into $\tilde{\mu}_t$:

[

$\tilde{\mu}_t$

$$\frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon \right)$$

\right).
]

Now define:

[
\epsilon_\theta(x_t,t)
]

and set:

[

\mu_\theta

\frac{1}{\sqrt{\alpha_t}}
\left(
x_t-
\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}
\epsilon_\theta
\right).
]

Then:

[

\boxed{

L_{t-1}

w_t
\mathbb{E}
[
|\epsilon-\epsilon_\theta|^2
]

$$+C.$$

$$\}$$

$$]$$

Finally, the authors remove (w_t) and use:

$$[$$

$$\boxed{\}$$

$$L_{\{\text{simple}\}}$$

$$\mathbb{E}$$

$$[$$

$$|\epsilon - \epsilon_{\theta}|^2$$

$$].$$

$$\}$$

$$]$$

That is the entire mathematical journey of Section 3.2–3.4.

60. What you should remember for understanding DDPM

If you don't want to memorize every derivation, remember these seven equations:

1. Forward one-step process

$$[$$

$$q(x_t|x_{t-1})$$

```

\mathcal{N}
(
\sqrt{\alpha_t}x_{t-1},
\beta_t
).
]

```

2. Direct noising

```

[
\boxed{
x_t=
\sqrt{\bar{\alpha}_t}x_0+
\sqrt{1-\bar{\alpha}_t}\epsilon
}
]

```

3. True posterior

[

$q(x_{t-1}|x_t, x_0)$

```

\mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t).
]

```

4. Reverse model

[

$p_{\theta}(x_{t-1}|x_t)$

```

\mathcal{N}(\mu_{\theta}, \sigma_t^2).
]

```

5. Noise parameterization

[

\boxed{

\mu_\theta

\frac{1}{\sqrt{\alpha_t}}
\left(
x_t-
\frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}
\epsilon_\theta(x_t,t)
\right)
}
]

6. Reverse sampling

[

\boxed{

x_{t-1}

\mu_\theta+\sigma_tz
}
]

7. Practical training loss

[

\boxed{

L_{\text{simple}}

```
\mathbb E
\left[
|\epsilon-\epsilon_{\theta(x_t,t)}|^2
\right].
}
]
```

61. The single most important intuition

Don't think of DDPM as:

"The neural network directly generates an image."

Instead think:

The neural network learns how to recognize and remove Gaussian noise at every possible noise level.

Then generation is simply:

```
[
\boxed{
\text{start with noise}
\rightarrow
\text{remove a little}
\rightarrow
\text{remove a little}
\rightarrow
\cdots
\rightarrow
\text{obtain data}.
}
]
```

And mathematically, the reason this works is that:

```
[
\boxed{
\text{noise prediction}
\leftrightharrow
\text{reverse mean prediction}
\leftrightharrow
\text{score estimation}
}
]
```

under the parameterization developed in this section.

So the progression of ideas in the paper is:

```
[
\boxed{
\begin{array}{c}
\text{Forward Gaussian diffusion}\backslash
\downarrow\backslash
\text{Variational inference / ELBO}\backslash
\downarrow\backslash
\text{Gaussian KL between forward posterior and reverse model}\backslash
\downarrow\backslash
\text{Predict posterior mean}\backslash
\downarrow\backslash
\text{Rewrite posterior mean using the actual noise }\epsilon\backslash
\downarrow\backslash
\text{Make the network predict }\epsilon\backslash
\downarrow\backslash
\text{Noise-prediction MSE}\backslash
\downarrow\backslash
\text{Denoising score matching interpretation}\backslash
\downarrow\backslash
\text{Langevin-like reverse sampling}\backslash
\downarrow\backslash
\text{Generation from Gaussian noise}
\end{array}
}
]
```

That is the core intellectual contribution of Sections 3.1–3.4.

Yes. Let's derive it purely mathematically.

Define

$$\left[\begin{array}{l} \alpha_t = 1 - \beta_t, \\ \text{\\quad} \\ \bar{\alpha}_t = \prod_{s=1}^t \alpha_s. \end{array} \right]$$

We want:

```
[
q(x_{t-1}\mid x_t,x_0).
]
```

1. Apply Bayes' rule

[
q(x_{t-1}\mid x_t, x_0)

```
\frac{
q(x_t\mid x_{t-1},x_0),
q(x_{t-1}\mid x_0)
}{
q(x_t\mid x_0)
}.
]
```

Using the Markov property,

[
q($x_t \mid x_{t-1}$, x_0)

$$q(x_t \mid x_{t-1}).$$

Therefore,

$$\begin{aligned} & q(x_{t-1} \mid x_t, x_0) \\ & \propto q(x_t \mid x_{t-1}) q(x_{t-1} \mid x_0). \end{aligned}$$

2. Substitute the two Gaussian distributions

The forward transition is

$$q(x_t \mid x_{t-1})$$

$$\begin{aligned} & \mathcal{N} \\ & \left(x_t; \right. \\ & \left. \sqrt{\alpha_t} x_{t-1}, \right. \\ & \left. \beta_t I \right) \end{aligned}$$

And the direct distribution from (x_0) is

$$q(x_{t-1} \mid x_0)$$

$$\begin{aligned} & \mathcal{N} \\ & \left(x_{t-1}; \right. \\ & \left. \sqrt{\bar{\alpha}_{t-1}} x_0, \right. \\ & \left. (1 - \bar{\alpha}_{t-1}) I \right) \end{aligned}$$

Therefore, ignoring constants independent of (x_{t-1}) ,

$$\begin{aligned} & \left[\begin{aligned} & q(x_{t-1} \mid x_t, x_0) \\ & \propto \exp \left[-\frac{1}{2\beta_t} \left(x_t - \sqrt{\alpha_t} x_{t-1} \right)^2 \right] \\ & \left[\begin{aligned} & \int \exp \left[-\frac{1}{2(1-\bar{\alpha}_{t-1})} \left(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}} x_0 \right)^2 \right] \right. \\ & \left. \right] \end{aligned} \end{aligned} \end{aligned}$$

3. Combine the exponentials

$$\begin{aligned} & \left[\begin{aligned} & q(x_{t-1} \mid x_t, x_0) \\ & \propto \exp \left\{ -\frac{1}{2} \left[\frac{\left(x_t - \sqrt{\alpha_t} x_{t-1} \right)^2}{\beta_t} + \frac{\left(x_{t-1} - \sqrt{\bar{\alpha}_{t-1}} x_0 \right)^2}{1 - \bar{\alpha}_{t-1}} \right] \right\} \end{aligned} \right] \end{aligned}$$

Expand only the terms containing (x_{t-1}) :

$$\begin{aligned} & \left[\right. \\ & \quad \exp\left\{ \right. \\ & \quad \quad -\frac{1}{2} \\ & \quad \quad \left[\right. \\ & \quad \quad \quad \frac{\alpha_t \beta_t}{\sigma^2} \\ & \quad \quad \quad + \\ & \quad \quad \quad \frac{1}{\sigma^2} (1 - \bar{\alpha}_{t-1}) x_{t-1} \\ & \quad \quad \left. \right] \\ & \quad \left. \right\} \\ & \quad \left. \right] x_{t-1}^2 \\ & \quad \left. \right] \end{aligned}$$

$$\left[\right.$$

$$\left. \right\}$$

$$+$$

$$\left[\right.$$

$$-\frac{\sqrt{\alpha_t} \beta_t}{\sigma^2} x_t$$

$$\begin{aligned} & \frac{\sqrt{\bar{\alpha}_{t-1}} \beta_{t-1}}{\sigma^2} (1 - \bar{\alpha}_{t-1}) x_0 \\ & \left. \right\}^T \\ & x_{t-1} \\ & \left. \right\}. \\ & \left. \right] \end{aligned}$$

This is a quadratic function of (x_{t-1}) .

A Gaussian density has exactly the form

$$\begin{aligned} & \left[\right. \\ & \quad \exp\left\{ \right. \\ & \quad \quad -\frac{1}{2} \\ & \quad \quad A |x_{t-1}|^2 \\ & \quad \quad + \\ & \quad \quad b^T x_{t-1} \end{aligned}$$

\right].
]

Therefore the posterior is Gaussian.

4. Complete the square

The resulting distribution is

[

\boxed{

$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$

\mathcal{N}
\left(
 $\mathbf{x}_{t-1};$
 $\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0),$
 $\tilde{\boldsymbol{\beta}}_t$
\right)
}
]

where

[

\boxed{

$\tilde{\boldsymbol{\beta}}_t$

$$\frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t} \beta_t$$

and

$$[$$

$$\boxed{}$$

$$\tilde{\mu}_t(x_t, x_0)$$

$$\frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1-\bar{\alpha}_t} x_0 + \frac{\sqrt{\alpha_t} (1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t} x_t$$

The key mathematical reason is simply:

$$\boxed{q(x_{t-1} \mid x_t, x_0) \propto \underbrace{q(x_t \mid x_{t-1})}_{\text{Gaussian}} \underbrace{q(x_{t-1} \mid x_0)}_{\text{Gaussian}}}$$

}
]

and the product of these two Gaussian functions is another Gaussian in (x_{t-1}) .

Yes, in this particular derivation, it is Gaussian.

From Bayes:

[

$q(x_{t-1} | x_t, x_0)$

$$\frac{q(x_t | x_{t-1})q(x_{t-1} | x_0)}{q(x_t | x_0)}$$

So schematically:

[
\boxed{
\frac{\text{Gaussian} \times \text{Gaussian}}{\text{Gaussian}}
}
]

and the result is Gaussian.

But there is an important subtlety:

"Gaussian $\times$ Gaussian $\div$ Gaussian is always Gaussian" is not true in complete generality.

What matters here is that the denominator

[
 $q(x_t | x_0)$
]

is a normalization constant with respect to (x_{t-1}) .

When we're finding the distribution of (x_{t-1}) , (x_t) and (x_0) are fixed. Therefore,

$$[q(x_t | x_0)]$$

doesn't depend on (x_{t-1}) .

So we can write

$$[q(x_{t-1} | x_t, x_0) \propto q(x_t | x_{t-1})q(x_{t-1} | x_0).]$$

Then it's simply

[

$\boxed{\text{Gaussian in } x_{t-1}}$

$\text{Gaussian in } x_{t-1}$

$\times$

$\text{Gaussian in } x_{t-1}$

$$\text{Gaussian in } x_{t-1}.$$

The denominator just normalizes that Gaussian so that its integral equals 1.

That's the cleanest way to think about the derivation.

Yes — with respect to (x_{t-1}) , it is a constant.

In

[

$q(x_{t-1} \mid x_t, x_0)$

$$\frac{q(x_t \mid x_{t-1})q(x_{t-1} \mid x_0)}{q(x_t \mid x_0)},$$

we are finding a probability distribution as a function of (x_{t-1}) .

At this point:

[
 $x_t = \text{fixed}, \text{ } \forall x_0 = \text{fixed}.$
]

Therefore,

[
 $q(x_t \mid x_0)$
]

depends only on (x_t) and (x_0) , not on (x_{t-1}) .

So, with respect to (x_{t-1}) ,

[
 $\boxed{q(x_t \mid x_0) = \text{constant}.}$
]

Hence we can write

[
 $q(x_{t-1} \mid x_t, x_0)$
 $\propto$
 $q(x_t \mid x_{t-1})q(x_{t-1} \mid x_0).$
]

And since both factors on the right are Gaussian functions of (x_{t-1}) ,

```
[
\boxed{
q(x_{t-1}|x_t, x_0)
\text{ is Gaussian}.
}
]
```

The denominator is basically the normalizing constant that makes the resulting Gaussian integrate to 1.

Yes — if by "Gaussian into Gaussian" you mean the product of two Gaussian density functions in the same variable, then the result is proportional to another Gaussian.

For example,

```
[
f(x)=\mathcal{N}(x;\mu_1,\sigma_1^2)
]
```

and

```
[
g(x)=\mathcal{N}(x;\mu_2,\sigma_2^2).
]
```

Then

```
[
\boxed{
f(x)g(x)\propto \mathcal{N}(x;\mu,\sigma^2)
}
]
```

because multiplying their exponential terms gives another quadratic exponent:

```
[
```

$e^{-a(x-\mu_1)^2}$

$$e^{\{-b(x-\mu_2)^2\}}$$

$$e^{\{-\left[a(x-\mu_1)^2+b(x-\mu_2)^2\right]\}},$$

and after completing the square, this is another Gaussian.

In your DDPM derivation

$$\begin{aligned} &[\\ & q(x_{t-1}|x_t, x_0) \\ & \propto \\ & \underbrace{q(x_t|x_{t-1})}_{\text{Gaussian in } x_{t-1}} \\ & \underbrace{q(x_{t-1}|x_0)}_{\text{Gaussian in } x_{t-1}}. \\ &] \end{aligned}$$

Therefore,

$$\begin{aligned} &[\\ & \boxed{ \\ & q(x_{t-1}|x_t, x_0) \\ & \text{ is Gaussian}. \\ & } \\ &] \end{aligned}$$

The important qualification is "in the same variable." Two arbitrary Gaussian distributions being multiplied isn't the relevant statement; their density functions must both be Gaussian functions of (x_{t-1}) .

Not exactly. "Gaussian in (x_{t-1}) " means that (x_{t-1}) is the variable whose density has the Gaussian form. It is not about whether it appears on the left or right of the conditioning bar.

For example,

[

$$q(x_{t-1} \mid x_0)$$

$$\mathcal{N}(x_{t-1}; \mu, \Sigma)$$

]

is Gaussian in (x_{t-1}) because the density is a Gaussian function of (x_{t-1}) .

But also consider

[

$q(x_t \mid x_{t-1})$

$\mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, \beta_t I)$.

]

Here (x_{t-1}) appears on the right of $(\mid)$, but this is also a Gaussian function of (x_{t-1}) when (x_t) is fixed:

[

$\mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, \beta_t I)$

$\propto$

$\exp\left(-\frac{1}{2\beta_t} \right.$

$\left. |x_t - \sqrt{\alpha_t} x_{t-1}|^2 \right)$

$\mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, \beta_t I)$

$\propto \exp\left(-\frac{1}{2\beta_t} \right.$

]

That expression is Gaussian-shaped as a function of (x_{t-1}) .

So in the DDPM derivation:

[

$q(x_t \mid x_{t-1})$

]

is Gaussian as a distribution of (x_t) , but it is also a Gaussian-shaped function of (x_{t-1}) when (x_t) is fixed.

That's why

[

$q(x_t \mid x_{t-1}) q(x_{t-1} \mid x_0)$

]

is a product of two Gaussian functions with respect to (x_{t-1}) .

Yes. That is exactly the right way to think about it. Just one small terminology correction.

1. First expression

[

$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_0)$$

$\mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$
]

is equivalent to saying

[
 $\boxed{\mathbf{x}_{t-1} \mid \mathbf{x}_0 \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})}$.
]

Here, $(\mathbf{x}_{t-1})$ is explicitly the random variable being sampled, so the Gaussian is naturally a density in $(\mathbf{x}_{t-1})$.

2. Second expression

The forward transition is

[

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

$\mathcal{N}$
 $\left(\right.$
 $\mathbf{x}_t;$
 $\sqrt{\alpha_t} \mathbf{x}_{t-1},$
 $\beta_t \mathbf{I}$
 $\left. \right)$.
]

Normally, this notation describes the distribution of (x_t) given (x_{t-1}) :

$$[\begin{array}{l} x_t | x_{t-1} \\ \sim \\ \mathcal{N} \\ \left(\begin{array}{l} \sqrt{\alpha_t} x_{t-1}, \\ \beta_t I \end{array} \right). \end{array}]$$

But when deriving

$$[\begin{array}{l} q(x_{t-1} | x_t, x_0), \end{array}]$$

we treat (x_t) and (x_0) as fixed and (x_{t-1}) as the variable.

Then

$$[\begin{array}{l} \mathcal{N} \\ \left(\begin{array}{l} x_t; \\ \sqrt{\alpha_t} x_{t-1}, \\ \beta_t I \end{array} \right) \end{array}]$$

is viewed as a function of (x_{t-1}) .

Explicitly,

$$[\begin{array}{l} \propto \\ \exp \left[\begin{array}{l} -\frac{1}{2\beta_t} \\ \left| \begin{array}{l} x_t - \sqrt{\alpha_t} x_{t-1} \end{array} \right|^2 \end{array} \right]. \end{array}]$$

Since (x_t) is fixed, this is a quadratic function of (x_{t-1}) , and therefore it has Gaussian form in (x_{t-1}) .

So your understanding is:

```
[
\boxed{
\text{The same Gaussian density can be viewed as a function of a different variable when
the other quantities are fixed.}
}
]
```

And that's exactly what allows

```
[
q(x_t \mid x_{t-1})q(x_{t-1} \mid x_0)
]
```

to be treated as

```
[
\boxed{\text{Gaussian function of } x_{t-1}}
\times
\text{Gaussian function of } x_{t-1}}
]
```

and hence the posterior

```
[
\boxed{q(x_{t-1} \mid x_t, x_0)}
]
```

is Gaussian.

Yes — but there is a subtle point: Rao–Blackwellization is not what makes the KL Gaussian. The Gaussian structure is what lets us calculate the KL in closed form. Rao–Blackwellization refers to avoiding/reducing Monte Carlo variance by analytically conditioning/averaging.

In DDPM, the logic is:

```
[
```

$$q(x_{t-1} \mid x_t, x_0)$$

$$\mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t)$$

and

[

$$p_{\theta}(x_{t-1} \mid x_t)$$

$$\mathcal{N}(\mu_{\theta}, \Sigma_{\theta}).$$

Therefore,

$$D_{\mathrm{KL}} \left[q(x_{t-1} \mid x_t, x_0) \parallel p_{\theta}(x_{t-1} \mid x_t) \right]$$

can be calculated analytically using the Gaussian-Gaussian KL formula.

Where Rao–Blackwellization enters

Suppose the ELBO contains an expectation like

$$\mathbb{E}_q \left[\text{some quantity} \right]$$

A naive approach could estimate it by sampling the entire trajectory

$$x_1, x_2, \dots, x_T$$

and averaging. That introduces Monte Carlo variance.

But because we know the conditional Gaussian structure, we can analytically evaluate certain conditional expectations instead of sampling those variables.

Schematically:

```
[
\boxed{
\text{Monte Carlo estimate}
\quad\longrightarrow\quad
\text{conditional expectation}
}
]
```

which reduces variance. That's the Rao–Blackwell idea.

In DDPM specifically

The important sequence is:

```
[
\boxed{
\text{ELBO}
\longrightarrow
\text{KL terms}
\longrightarrow
q(x_{t-1}|x_t,x_0);p_{\theta}(x_{t-1}|x_t)
}
]
```

Then:

```
[
\boxed{
\text{both Gaussian}
\longrightarrow
\text{KL has closed form}
\longrightarrow
\text{no sampling needed for that KL}
}
]
```

So if your paper says the objective is Rao–Blackwellized, don't interpret that as:

"Rao–Blackwellization makes the Gaussian KL easy."

Instead:

The ELBO can be rewritten/conditioned so that quantities that could otherwise be estimated with noisy Monte Carlo sampling are evaluated analytically; in DDPM, the resulting Gaussian KL terms are especially easy because they have closed-form formulas.

And this is why DDPM training can ultimately reduce to a very simple noise-prediction objective such as

$$\boxed{\mathbb{E}_{\mathbf{x}_0, t, \epsilon} \left[\|\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)\|^2 \right]}$$

That simple loss is a consequence of the ELBO/KL derivation plus the Gaussian diffusion structure.

Closed form means you can calculate the answer directly using a mathematical formula, without needing an iterative procedure or repeatedly sampling random values.

For example, the Gaussian KL has the closed-form formula

[

$$D_{\mathrm{KL}}(\mathcal{N}_1 \| \mathcal{N}_2)$$

$$\frac{1}{2} \left[\log \frac{|\Sigma_2|}{|\Sigma_1|} - \frac{1}{2} \operatorname{tr}(\Sigma_2^{-1} \Sigma_1) + \frac{1}{2} (\mu_2 - \mu_1)^T \Sigma_2^{-1} (\mu_2 - \mu_1) \right]$$

So if you know

```
[  
\mu_1,\Sigma_1,\mu_2,\Sigma_2,  
]
```

you just plug them into the formula and get the KL.

Contrast with Monte Carlo

Suppose you want

```
[  
\mathbb{E}_{x \sim q}[f(x)].  
]
```

Without a closed-form solution, you might:

```
[  
x_1,x_2,\ldots,x_N \sim q  
]
```

and estimate

```
[  
\mathbb{E}_q[f(x)]  
\approx  
\frac{1}{N} \sum_{i=1}^N f(x_i).  
]
```

That's a Monte Carlo estimate.

So:

```
[  
\boxed{\text{Closed form}=\text{direct formula}}  
]
```

```
[  
\boxed{\text{Monte Carlo}=\text{sample and estimate}}  
]
```

In DDPM, because the relevant distributions are Gaussian, the KL can be computed directly, rather than estimated by sampling (x_{t-1}) many times.

Because we generally cannot calculate the exact minimum of $(-\log p_{\theta}(x))$ directly.

The key distinction is between the quantity we want and a tractable objective we can optimize.

We want to maximize the data likelihood:

$$\left[\log p_{\theta}(x) \right]$$

or equivalently minimize

$$\left[-\log p_{\theta}(x) \right]$$

The problem in a diffusion model is that

$$\left[\right]$$

$$p_{\theta}(x_0)$$

$$\int p_{\theta}(x_{0:T}) dx_{1:T}.$$

The latent variables

$$\left[x_1, \dots, x_T \right]$$

must be integrated out.

So

$$\left[\right]$$

$$-\log p_{\theta}(x_0)$$

$$-\log \int p_{\theta}(x_{0:T}) dx_{1:T}$$

is not directly tractable.

So we introduce a bound

Using the variational distribution ($q(x_{1:T}|x_0)$), we get

$$-\log p_{\theta}(x_0) \leq \mathrm{ELBO}$$

or equivalently,

$$\log p_{\theta}(x_0) \geq \mathrm{ELBO}.$$

So instead of minimizing the difficult quantity

$$-\log p_{\theta}(x_0),$$

we minimize its tractable upper bound

$$-\mathrm{ELBO}.$$

Why is this useful?

Because the ELBO can be decomposed into terms such as

$$D_{\mathrm{KL}}$$

```

\left(
q(x_{t-1}|x_t,x_0)
|
p_{\theta}(x_{t-1}|x_t)
\right),
]

```

and these are tractable because the distributions are Gaussian.

So the whole logic is:

```

[
\boxed{
\text{Want}
\quad
\min_{\theta}[-\log p_{\theta}(x_0)]
}
]

```

but

```

[
\boxed{
-\log p_{\theta}(x_0)
\text{ is intractable}
}
]

```

therefore

```

[
\boxed{
\text{derive a tractable bound}
}
]

```

and optimize

```

[
\boxed{
\min_{\theta}[-\mathrm{ELBO}].
}
]

```

One important point

We are not saying the bound is the actual minimum of $(-\log p_{\theta}(x))$.

We are saying:

The exact objective is difficult to compute, so we optimize a tractable bound on it.

And if the bound is tight, optimizing the bound closely corresponds to optimizing the true likelihood.

Exactly — a bound by itself does not magically give you the minimum of the true $(-\log p_{\theta}(x))$. The reason we use it is more specific:

We cannot evaluate the true objective easily, so we optimize a computable surrogate that is guaranteed to be related to it.

Let's make the distinction precise.

1. What we actually want

Maximum likelihood means

[

θ^*

$\arg\min_{\theta} \left[-\log p_{\theta}(x) \right]$.
]

If we could calculate this quantity and its gradient efficiently, we would directly minimize it.

But in a diffusion model,

[

$p_{\theta}(x_0)$

$$\int p_{\theta}(x_{0:T}) dx_{1:T}.$$

Therefore,

$$[$$

$$-\log p_{\theta}(x_0)$$

$$-\log \int p_{\theta}(x_{0:T}) dx_{1:T}.$$

That integral is over potentially thousands/millions of latent dimensions and is intractable.

So we cannot practically evaluate

$$[-\log p_{\theta}(x_0)]$$

for a given θ , which means we cannot practically compute its gradient and run ordinary gradient descent on it.

2. The ELBO gives us something we CAN calculate

We derive

$$[-\log p_{\theta}(x_0) \geq \text{ELBO}(\theta).]$$

Equivalently,

$$[-\log p_{\theta}(x_0) \leq$$

$-\mathrm{ELBO}(\theta).$

So we minimize

$$-\mathrm{ELBO}$$

because this quantity can actually be evaluated and differentiated.

And importantly, it decomposes into tractable Gaussian KL terms.

3. Does minimizing the bound guarantee the exact minimum?

Not necessarily.

This is the important point you're getting at.

Suppose

$$f(\theta) \leq g(\theta).$$

Minimizing (g) does not generally mean you have found the minimum of (f) .

So yes, your concern is mathematically valid.

What makes variational inference useful is that the bound is connected to the exact objective through

$$[$$

$$\mathbb{E}_{q(\theta)}$$

$$\log p_{\theta}(x)$$

$$\begin{aligned} & \mathrm{ELBO} \\ & + \\ & D_{\mathrm{KL}} \\ & \left(\right. \\ & q(z|x) | p_{\theta}(z|x) \\ & \left. \right). \\ & \} \\ &] \end{aligned}$$

Since

$$\begin{aligned} & [\\ & D_{\mathrm{KL}} \geq 0, \\ &] \end{aligned}$$

ELBO is a lower bound.

If the variational posterior (q) becomes equal to the true posterior,

$$\begin{aligned} & [\\ & q(z|x) = p_{\theta}(z|x), \\ &] \end{aligned}$$

then

$$\begin{aligned} & [\\ & D_{\mathrm{KL}} = 0 \\ &] \end{aligned}$$

and therefore

$$\begin{aligned} & [\\ & \boxed{\mathrm{ELBO} = \log p_{\theta}(x).} \\ &] \end{aligned}$$

So the bound can become tight.

4. Why DDPM is particularly convenient

DDPM chooses a forward process (q) whose conditional distributions are Gaussian and analytically tractable.

Therefore the ELBO can be decomposed into terms like

```
[
D_{\mathrm{KL}}
\left(
q(x_{t-1}|x_t,x_0)
|
p_{\theta}(x_{t-1}|x_t)
\right),
]
```

which we can calculate exactly.

So the practical reasoning is:

```
[
\boxed{
\text{True likelihood}
\rightarrow
\text{intractable}
}
]
```

```
[
\downarrow
]
```

```
[
\boxed{
\text{ELBO}
\rightarrow
\text{tractable}
}
]
```

```
[
\downarrow
]
```

```
[
\boxed{
\text{optimize ELBO with gradient descent}
}
]
```

So we aren't using the bound because a bound somehow makes finding the true minimum easier.

We're using it because:

```
[
\boxed{
\text{the exact objective is inaccessible, while the bound is computable and
differentiable.}
}
]
```

And that's the fundamental reason variational inference is needed in these latent-variable models.

Yes, partly, but I would phrase it more precisely.

A bound can be used in two ways:

1. As information about the true quantity's range

```
[
\mathrm{ELBO} \leq \log p_{\theta}(x)
]
```

tells us that the true log-likelihood is at least the ELBO.
2. As a surrogate objective that we optimize directly
In variational inference, we often cannot conveniently optimize the true

```
[
\log p_{\theta}(x),
]
```

so we optimize the ELBO instead.

The important point is that the ELBO is not merely used to "understand the range." It is often deliberately used as the training objective because it is tractable.

For DDPM:

```
[
\boxed{
\log p_{\theta}(x_0)
\geq
\mathrm{ELBO}
}
]
```

and therefore

```
[
\boxed{
-\log p_{\theta}(x_0)
}
```

$\leq$
 $\mathbb{E}_{q(\mathbf{z})}[\log p_{\theta}(\mathbf{x}_T | \mathbf{x}_0)]$.
 $\int$
 $\mathbb{E}_{q(\mathbf{z})}[\log p_{\theta}(\mathbf{x}_T | \mathbf{x}_0)]$

We cannot conveniently calculate the left side because

[

$p_{\theta}(\mathbf{x}_0)$

$\int p_{\theta}(\mathbf{x}_T | \mathbf{x}_0) d\mathbf{x}_T$
 $\int$

requires an intractable integral over the latent diffusion states.

But the ELBO can be decomposed into tractable terms, including Gaussian KL divergences.

So we use:

$\mathbb{E}_{q(\mathbf{z})}[\log p_{\theta}(\mathbf{x}_T | \mathbf{x}_0)]$
 $\mathbb{E}_{q(\mathbf{z})}[\log p_{\theta}(\mathbf{x}_T | \mathbf{x}_0)]$
 $\mathbb{E}_{q(\mathbf{z})}[\log p_{\theta}(\mathbf{x}_T | \mathbf{x}_0)]$

And if the bound becomes tight,

$\mathbb{E}_{q(\mathbf{z})}[\log p_{\theta}(\mathbf{x}_T | \mathbf{x}_0)]$
 $\mathbb{E}_{q(\mathbf{z})}[\log p_{\theta}(\mathbf{x}_T | \mathbf{x}_0)]$
 $\mathbb{E}_{q(\mathbf{z})}[\log p_{\theta}(\mathbf{x}_T | \mathbf{x}_0)]$

then the surrogate and true objective are exactly the same.

So your intuition is right, but "we use the bound to understand its range" is only one role; in DDPM, its main practical role is that we can optimize the bound directly when the true likelihood is intractable.

Yes — that is a good intuition, with one important refinement.

We have the true quantity:

$$\left[\log p_{\theta}(x) \right]$$

which we want to maximize, but it is difficult to compute.

So we construct a tractable quantity:

$$\left[\mathrm{ELBO}(\theta) \right]$$

such that

$$\boxed{\mathrm{ELBO}(\theta) \leq \log p_{\theta}(x)}$$

We then study and optimize the ELBO as a proxy for the behavior of the true log-likelihood.

The key relationship is actually:

$$\boxed{\log p_{\theta}(x)}$$

$$\begin{aligned} & \mathrm{ELBO}(\theta) \\ & + \\ & D_{\mathrm{KL}} \left(q(z|x) \parallel p_{\theta}(z|x) \right) \end{aligned}$$

Since

$$[D_{\mathrm{KL}} \geq 0,$$

the ELBO is always below the true log-likelihood.

So your intuition can be stated as:

We cannot conveniently observe/optimize the true ($\log p_{\theta}(x)$), so we construct a computable lower bound whose behavior is related to it, and optimize that bound instead.

And if

$$[D_{\mathrm{KL}} \rightarrow 0,$$

then

$$[\mathrm{ELBO} \rightarrow \log p_{\theta}(x),$$

so the proxy becomes an exact representation of the true objective.

That's essentially the motivation behind variational inference.

Correct. In the original DDPM formulation, the covariance is not necessarily predicted by the neural network.

The reverse process is

[

$$p_{\theta}(x_{t-1} \mid x_t)$$

$$\begin{aligned} & \mathcal{N} \\ & \left(\right. \\ & x_{t-1}; \\ & \mu_{\theta}(x_t, t), \\ & \Sigma_{\theta}(x_t, t) \end{aligned}$$

\right).
]

There are two separate quantities:

[
\boxed{\mu_{\theta}(x_t,t)}
]

and

[
\boxed{\Sigma_{\theta}(x_t,t)}.
]

In the original DDPM

Ho et al. choose the covariance to be fixed, rather than learned:

[
\boxed{
\Sigma_{\theta}(x_t,t)=\sigma_t^2 I
}
]

where (σ_t^2) is determined by the diffusion schedule.

So the neural network primarily predicts the information needed to determine the mean.

In the common (ϵ) -prediction parameterization:

[
(x_t,t)
\overset{\text{NN}}{\longrightarrow}
\epsilon_{\theta}(x_t,t)
]

then

[
\epsilon_{\theta}
\longrightarrow
\mu_{\theta}(x_t,t).
]

The covariance is already specified:

```
[
\Sigma_\theta=\sigma_t^2].
]
```

But other diffusion models can learn the variance

More generally, you can let the network predict both:

```
[
\boxed{
(\mu_\theta,\Sigma_\theta)
}
]
```

or an equivalent parameterization of the variance.

So your statement is right:

When a paper says "choose $(\Sigma_\theta(x_t,t))$," it does not necessarily mean the neural network predicts it. It means the model designer must specify/parameterize the reverse covariance.

For basic DDPM:

```
[
\boxed{
\text{NN predicts noise/mean; covariance is fixed.}
}
]
```

Yes — these two are easy to confuse because they represent different means.

1. $(\tilde{\mu}_t)$ — the true posterior mean

This belongs to the forward process's posterior:

```
[
```

```
\boxed{
```


$$q(\mathbf{x}_{t-1} \mid \mathbf{x}_t, \mathbf{x}_0)$$

$$\mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t, \tilde{\boldsymbol{\beta}}_t)$$

where

[

$\boxed{}$

$\tilde{\boldsymbol{\mu}}_t$

$$\frac{\sqrt{\bar{\alpha}_{t-1}} \beta_t}{1 - \bar{\alpha}_t} \mathbf{x}_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t} \mathbf{x}_t$$

This is known mathematically during training, because $(\mathbf{x}_0)$, $(\mathbf{x}_t)$, and the noise schedule are known.

2. $(\boldsymbol{\mu}_\theta)$ — the learned reverse mean

This belongs to the model:

[

$\boxed{}$

$p_{\theta}(x_{t-1} \mid x_t)$

$\mathcal{N}(x_{t-1}; \mu_{\theta}, \Sigma_{\theta})$
}
]

The neural network determines (μ_{θ}) , often by predicting noise:

[
 $\epsilon_{\theta}(x_t, t)$
]

and then

[

$\boxed{}$

μ_{θ}

$\frac{1}{\sqrt{\alpha_t}}$
 $\left($
 $x_t -$
 $\frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t)$
 $\right).$
}
]

The important distinction

[

$\boxed{}$

$\tilde{\mu}_t$

$\text{true posterior mean}$
}
]

[

$\boxed{}$

μ_θ

$\text{learned reverse-process mean}$
}
]

During training, we want them to become close:

[
 $\boxed{}$
 $\mu_\theta(x_t, t)$
 $\approx$
 $\tilde{\mu}_t(x_t, x_0)$
]
]

because that makes

[
 $p_\theta(x_{t-1} \mid x_t)$
]

behave like the true posterior

```
[
q(x_{t-1} \mid x_t, x_0).
]
```

So you can think:

```
[
\underbrace{\tilde{\mu}_t}_{\text{teacher / known}}
\quad \longleftrightarrow \quad
\underbrace{\mu_{\theta}_t}_{\text{model / learned}}.
]
```

The paper is saying that autoregressive models are actually a special case of the general diffusion/variational framework.

Normally, diffusion gradually adds noise. But imagine a different forward process:

```
[
x_0=(x_1,x_2,\ldots,x_T)
]
```

and at step (t), we mask the (t)-th coordinate.

So:

```
[
x_0
\rightarrow
x_1
\rightarrow
x_2
\rightarrow \cdots \rightarrow
x_T
]
```

gradually becomes a blank object.

What does the reverse model do?

At reverse step (t), it receives

```
[
x_t=(\text{blank first } t,; x_{t+1}, \ldots, x_T)
]
```

and has to reconstruct (x_t)'s missing coordinate.

Therefore,

```
[  
p_\theta(x_{t-1} \mid x_t)  
]
```

learns to:

1. copy coordinates $(t+1, \dots, T)$ unchanged;
2. predict coordinate (t) from coordinates $(t+1, \dots, T)$.

So it learns something like

```
[  
\boxed{  
p_\theta(x_t \mid x_{t+1:T})  
}  
]
```

which is exactly what an autoregressive model does.

For example, generating an image/vector backwards:

```
[  
x_T  
\rightarrow x_{T-1}  
\rightarrow x_{T-2}  
\rightarrow \cdots  
\rightarrow x_0  
]
```

means:

```
[  
p(x_T)  
p(x_{T-1} \mid x_T)  
p(x_{T-2} \mid x_{T-1:T})  
\cdots  
p(x_1 \mid x_{2:T}).  
]
```

That's an autoregressive factorization.

So the paper's point is:

```
[
\boxed{\text{Diffusion framework can include autoregressive modeling as a special
case.}}
]
```

The difference is that ordinary diffusion masks/adds noise gradually, whereas this special construction masks one coordinate at a time.

Sure. The common idea behind all three is progressive refinement: information is revealed or reconstructed in stages rather than all at once.

1. Progressive generation

The model generates an object step by step, where each step improves the previous one.

For diffusion:

```
[
\text{pure noise}
\rightarrow
\text{rough structure}
\rightarrow
\text{recognizable image}
\rightarrow
\text{fine details}
]
```

Each intermediate state is already a partially meaningful version of the final sample.

2. Progressive coding

Information is encoded in successive stages.

For example, imagine encoding an image:

```
[
\text{coarse structure}
\rightarrow
+\text{edges}
\rightarrow
+\text{textures}
\rightarrow
+\text{fine details}
]
```

The first part of the code gives a rough description, while later parts progressively refine it.

So the receiver doesn't necessarily need the entire code to understand the basic content.

3. Progressive lossy compression

Here the goal is to compress data while allowing different quality levels.

Instead of sending one complete compressed representation, we can send:

```
[  
  \boxed{  
    \text{low quality}  
    \rightarrow  
    \text{medium quality}  
    \rightarrow  
    \text{high quality}  
  }  
]
```

For example, an image might initially be reconstructed at (25%) quality, then additional information improves it to (50%), (75%), and finally (100%).

The important point is that you can stop at any stage if the current quality is sufficient.

The common idea

All three follow:

```
[  
  \boxed{  
    \text{coarse information}  
    \rightarrow  
    \text{progressive refinement}  
    \rightarrow  
    \text{full information}  
  }  
]
```

The difference is what is being progressively refined:

Concept	What is progressively refined?
Progressive generation	The generated sample
Progressive coding	The representation/code
Progressive lossy compression	The reconstruction quality

This is one reason diffusion is sometimes viewed as more than just "adding and removing noise": it provides a sequence of progressively refined representations of the data.

Yes, roughly, but one correction: we don't necessarily throw bits away *to make space for the new bits*. The idea is that each additional stage gives more information while staying within a chosen bit budget/rate.

Think of it like:

```
[
\text{Original image}
]
```

↓

```
[
\text{Stage 1: coarse approximation}
]
```

↓

```
[
+\text{additional information}
]
```

```
[
\text{Stage 2: better approximation}
]
```


↓

```
[  
+ \text{additional information}  
]
```

```
[  
\text{Stage 3: even better approximation}  
]
```

Because it's lossy, some original information is discarded/approximated rather than preserved exactly.

If there is a fixed storage/transmission budget

Then your intuition becomes especially appropriate:

```
[  
\boxed{  
\text{add useful information}  
\quad+\quad  
\text{discard/approximate less important information}  
}  
]
```

So you can think:

Progressive lossy compression = progressively improve the reconstruction by sending more information, while accepting that some information from the original is permanently lost.

The key word is **lossy**: the final reconstruction does not necessarily contain all the original information.

Yes — your explanation is correct. The key point is the distinction between a continuous probability density and the probability assigned to a discrete pixel value.

1. Why can't we just use $N(127; \mu, \sigma^2)$?

A Gaussian gives a density:

$$p(x) = N(x; \mu, \sigma^2).$$

So

$$N(127; \mu, \sigma^2)$$

means the height of the Gaussian density at $x=127$.

It is not

$P(x=127)$.

In fact, for a continuous random variable,

$P(x=127)=0$.

The probability comes from an interval:

$P(126.5 \leq x < 127.5) = \int_{126.5}^{127.5} p(x) dx$.

2. Why use 126.5 to 127.5?

Because the actual image contains a discrete integer value:

$x_{0i}=127$.

We can associate every integer pixel value with a bin of width 1:

$127 \leftrightarrow [126.5, 127.5)$.

Similarly:

$126 \leftrightarrow [125.5, 126.5)$ $128 \leftrightarrow [127.5, 128.5)$.

So the decoder takes the continuous Gaussian and integrates it over the bin corresponding to the observed pixel value.

3. What the equation in the paper is doing

The paper writes something like

$p_{\theta}(x_0 | x_1) = \sum_i \mathbb{1}_{[D_i - (x_{0i}) D_i + (x_{0i})]} N(x; \mu_{\theta, i}(x_1, 1), \sigma^2)$.

For one pixel $x_{0i}=127$, this becomes approximately

$\int_{126.5}^{127.5} N(x; \mu_{\theta, i}, \sigma^2) dx$.

So the model is asking:

How much probability mass does the predicted Gaussian assign to the observed pixel value 127?

That is the correct likelihood for the discrete observation.

4. Why the product appears

An image has D dimensions/pixels.

Assuming the decoder factorizes across dimensions,

$$p_{\theta}(x_0 | x_1) = \prod_{i=1}^D p_{\theta}(x_{0i} | x_1).$$

Each pixel contributes its own probability:

$$p_{\theta}(x_{0i} | x_1) = \int_{\text{pixel bin}} N(x; \mu_i, \sigma^2) dx.$$

Therefore the entire image likelihood is the product of the individual pixel probabilities.

In log form, which is what is normally optimized,

$$\log p_{\theta}(x_0 | x_1) = \sum_{i=1}^D \log \left[\int_{\text{pixel bin}} N(x; \mu_i, \sigma^2) dx \right].$$

The intuition

Think of it this way:

Gaussian density $\downarrow$ integrate over each pixel's bin $\downarrow$ probability of the observed discrete pixel

So the final decoder is essentially converting the continuous Gaussian prediction into a probability distribution over discrete pixel values.

One small nuance: for boundary pixel values such as 0 and 255, the bins are clipped to the valid range rather than simply using $x \pm 0.5$ without modification.

Exactly. Your point is correct.

The final decoder does not necessarily output an integer pixel value.

There are two separate concepts here:

1. What the decoder predicts

The decoder predicts the parameters of a Gaussian:

[
 $\mu_{\theta}(x_1), \propto \sigma^2.$
]

So its underlying prediction is a continuous distribution:

[
 $x_0 \sim \mathcal{N}(\mu_{\theta}, \sigma^2).$
]

If you actually sample from this Gaussian, you can get

[
 126.73, $\propto$ 127.41, $\propto$ 128.02, $\dots$
]

These are not necessarily integers.

2. What the final decoder likelihood represents

The discretized likelihood says:

"Instead of treating 126.73 and 127.41 as different pixel values, which discrete pixel value's bin do they belong to?"

For example,

[
 $[126.5, 127.5) \rightarrow \text{pixel value } 127.$
]

Thus

[

$P(x_0=127)$

$\int_{126.5}^{127.5}$
 $\mathcal{N}(x; \mu, \sigma^2), dx.$
]

This is used to calculate the likelihood of the observed integer pixel.

So the crucial distinction is:

[
 $\boxed{\text{Discretized likelihood} \neq \text{integer-valued decoder output}}$
]

The discretization is primarily about how the likelihood is calculated, not about forcing the neural network's raw output to be an integer.

Then how do we get an actual integer image?

If you want to generate an image, you need to convert the continuous sample into a discrete pixel value.

Conceptually:

[
 $x \sim \mathcal{N}(\mu, \sigma^2)$
]

then

[
 $\boxed{x \rightarrow \text{corresponding pixel bin}}$
]

For example:

[
 $127.42 \rightarrow 127$
]

[
 $128.31 \rightarrow 128.$
]

So yes: simply running the final decoder does not magically make the neural network output integers. The paper's final decoder is primarily defining a proper discrete-data likelihood for training/evaluation.

This distinction is very important in understanding DDPMs.

Yes — during the derivation/training of DDPMs, (x_{t-1}) can be sampled from

$$\boxed{q(x_{t-1} \mid x_t, x_0)}$$

but there is an important distinction between training and generation.

During training

We know the original clean image (x_0) .

The forward process is

$$[$$

$$q(x_t \mid x_{t-1})$$

$$\mathcal{N}(\sqrt{1-\beta_t}x_{t-1}, \beta_t I).$$

Given (x_0) and (x_t) , we can derive the posterior

$$[$$

$$\boxed{$$

$$q(x_{t-1} \mid x_t, x_0)$$

$$\mathcal{N}(\tilde{\mu}_t(x_t, x_0), \tilde{\beta}_t I)$$

and therefore sample

[

$\boxed{\cdot}$

x_{t-1}

$$\begin{aligned} & \tilde{\mu}_t(x_t, x_0) \\ & + \\ & \sqrt{\tilde{\beta}_t} z \\ & \end{aligned}$$

where

$$z \sim \mathcal{N}(0, I).$$

So yes, (x_{t-1}) is sampled from that posterior when we are analyzing/using the forward process with (x_0) known.

But during generation

We don't know (x_0) . That's the whole point.

We start with

$$x_T \sim \mathcal{N}(0, I)$$

and use the learned reverse model

$$\boxed{p_{\theta}(x_{t-1} \mid x_t)}$$

to sample:

[

x_{t-1}

$\mu_{\theta}(x_t, t) + \sigma_t z$.

]

So:

[

$\boxed{\text{Training:}}$

$q(x_{t-1} \mid x_t, x_0)$

}

]

[

$\boxed{\text{Generation:}}$

$p_{\theta}(x_{t-1} \mid x_t)$

}

]

The model is trained to approximate the true posterior

[

$q(x_{t-1} \mid x_t, x_0)$

]

using only information available during generation:

[

$p_{\theta}(x_{t-1} \mid x_t)$.

]

That distinction is fundamental in DDPMs.

Exactly. During generation/sampling, we sample from $(p_{\theta}(x_{t-1} \mid x_t))$, not from $(q(x_{t-1} \mid x_t, x_0))$.

The distinction is:

During training

We have access to the real clean image (x_0), so we can use the true posterior

$$[q(x_{t-1} | x_t, x_0).]$$

This tells us what the correct reverse step should look like.

During generation

We do not have (x_0) . We only have (x_t) .

Therefore, we cannot use

$$[q(x_{t-1} | x_t, x_0).]$$

Instead, the neural network learns an approximation:

$$[\boxed{p_{\theta}(x_{t-1} | x_t)}]$$

and we sample from it:

$$[\boxed{\begin{aligned} &x_{t-1} \\ &\sim \\ &p_{\theta}(x_{t-1} | x_t) \end{aligned}}]$$

or equivalently,

[

$$x_{t-1}$$

$$\sim \mathcal{N}(x_t, \sigma_t^2).$$

]

So the complete idea is:

```
[
\boxed{
q(x_{t-1}|x_t, x_0)
\quad\text{= true posterior used during training/derivation}
}
]
```

```
[
\boxed{
p_{\theta}(x_{t-1}|x_t)
\quad\text{= learned reverse distribution used during generation}
}
]
```

And (ϵ_{θ}) helps the model construct the parameters of this learned reverse distribution, particularly (μ_{θ}) .

In one sentence:

(q) tells us what the reverse step should be when (x_0) is known; (p_{θ}) is what we actually use to generate when (x_0) is unknown.

Yes, but with an important distinction.

In DDPMs, (q) is the fixed forward diffusion process, and it also appears as the variational/inference distribution in the ELBO formulation.

The relationship

The generative model is

```
[
```

$p_{\theta}(x_{0:T})$

```
p(x_T)
\prod_{t=1}^T
p_{\theta}(x_{t-1}|x_t).
]
```

This is the model we ultimately want to learn.

The forward process is

[

$q(x_{1:T} \mid x_0)$

$\prod_{t=1}^T$
 $q(x_t \mid x_{t-1}).$
]

We choose this (q) ourselves; it gradually adds Gaussian noise.

Because directly computing the posterior of the generative model is difficult, we use (q) as the variational distribution in the ELBO:

[
 $\log p_{\theta}(x_0)$
 $\geq$
 $\mathbb{E}_q$
 $\left[\log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T} \mid x_0)} \right].$
]

So yes:

[
 $\boxed{q \text{ acts as the variational/inference distribution}}$
]

while

[
 $\boxed{p_{\theta} \text{ is the learned generative/reverse model}}$.
]

But don't confuse these two (q)'s conceptually

The forward process is

[
 $q(x_t \mid x_{t-1}),$
]

which adds noise.

From this forward process, we can mathematically derive

$$[q(x_{t-1}|x_t, x_0),]$$

which is the true posterior of the forward diffusion chain.

During training, this posterior gives us the target that the learned reverse process should approximate:

$$[\boxed{p_{\theta}(x_{t-1}|x_t) \approx q(x_{t-1}|x_t, x_0)}]$$

But during sampling, (x_0) is unavailable, so we cannot use the right-hand side.

Therefore:

$$[\boxed{\text{Training: use } q(x_{t-1}|x_t, x_0)}]$$

$$[\boxed{\text{Generation: sample } p_{\theta}(x_{t-1}|x_t)}]$$

So your understanding is right: (q) is the variational/inference side of the DDPM formulation, while (p_{θ}) is the learned generative side.

Yes — your correction is important. Saying simply “ (q) is variational inference of (p) ” is misleading in DDPMs.

The more precise statement is:

$$\boxed{q(x_{1:T} \mid x_0)}$$

is a fixed forward diffusion process that we choose.

It is used as the variational distribution in the ELBO derivation, but it is not "the variational inference of p_θ " in the sense that q is obtained by performing inference on p_θ .

Why?

In ordinary variational inference, you might have an intractable posterior

$$p_\theta(z \mid x)$$

and introduce a parameterized approximation

$$q_\phi(z \mid x) \approx p_\theta(z \mid x).$$

Here, (q_ϕ) is learned to approximate the posterior.

DDPM is different.

We define:

[

$$q(x_t \mid x_{t-1})$$

$$\mathcal{N}(\sqrt{\alpha_t}x_{t-1}, \beta_t)$$

ourselves.

There is no learned (ϕ) here.

Then this fixed (q) gives us the analytically tractable posterior

```
[  
q(x_{t-1}|x_t, x_0).  
]
```

The neural network learns

```
[  
p_{\theta}(x_{t-1}|x_t)  
]
```

to approximate that reverse transition.

So a much better mental model is:

```
[  
\boxed{  
q=\text{fixed forward/noising process}  
}  
]
```

```
[  
\boxed{  
q(x_{t-1}|x_t, x_0)  
=\text{known reverse posterior of that forward process}  
}  
]
```

```
[  
\boxed{  
p_{\theta}(x_{t-1}|x_t)  
=\text{learned reverse/generative process}  
}  
]
```

And (q) is used as the variational distribution when deriving the ELBO, which is why DDPMs are connected to variational inference.

So your statement:

“(q) is not variational inference of (p)”

is correct if by that you mean (q) is not obtained by inferring the posterior of (p_{\theta}). It is a deliberately constructed distribution that is then used in the variational bound.

Yes. In the standard DDPM, $(p_{\theta}(x_{t-1} \mid x_t))$ is modeled as a Gaussian.

Specifically,

[

$\boxed{\{$

$p_{\theta}(x_{t-1} \mid x_t)$

```
\mathcal{N}
\left(
x_{t-1};
\mu_{\theta}(x_t, t),
\Sigma_{\theta}(x_t, t)
\right)
}
```

Often DDPM fixes the covariance to a known value, so you commonly see

[

$\boxed{\{$

$p_{\theta}(x_{t-1} \mid x_t)$

```
\mathcal{N}
\left(
x_{t-1};
\mu_{\theta}(x_t, t),
\sigma_t^2 I
\right)
```

$$\}$$

$$]$$

The neural network predicts the mean-related quantity, often through noise prediction:

$$[\epsilon_{\theta}(x_t, t)$$

$$]$$

and then

$$[$$

$$\mu_{\theta}(x_t, t)$$

$$\frac{1}{\sqrt{\alpha_t}}$$

$$\left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right).$$

$$]$$

Then we actually sample:

$$[$$

$$\boxed{\mu_{\theta}(x_t, t) + \sigma_t z,$$

$$x_{t-1}]$$

$$\mu_{\theta}(x_t, t) + \sigma_t z,$$

$$\quad z \sim \mathcal{N}(0, I)$$

$$\}$$

$$]$$

So the whole reverse step is:

```
[
x_t
\overset{\epsilon_\theta}{\longrightarrow}
\mu_\theta
]
```

then

```
[
\boxed{
x_{t-1} \sim
\mathcal{N}(\mu_\theta, \sigma_t^2)
}
]
```

Compare (q) and (p_θ)

Both are Gaussian in the standard DDPM:

```
[
```

```
\boxed{
```

q(x_t | x_{t-1})

```
\mathcal{N}(\sqrt{\alpha_t}x_{t-1}, \beta_t)
}
]
```

and

```
[
```

```
\boxed{
```

$p_{\theta}(x_{t-1} \mid x_t)$

$N(\mu_{\theta}, \sigma^2)$
}
]

But their roles are different:

- q → fixed forward process, adds noise.
- p_{θ} → learned reverse process, removes noise.
- ϵ_{θ} → neural network's prediction of the noise, used to determine the reverse Gaussian's mean.

So yes: p_{θ} is Gaussian; the network is essentially learning how to position that Gaussian at every timestep.

Yes — with one important qualification.

For the reverse step:

[
 $q(x_{t-1} \mid x_t, x_0)$
]

is the true/analytically known posterior of the forward diffusion process, because q is fixed and x_0 is known during training.

The model learns:

[
 $p_{\theta}(x_{t-1} \mid x_t)$
]

which is the predicted/learned reverse distribution.

So:

[
 $q(x_{t-1} \mid x_t, x_0)$
 $\quad \text{= true posterior}$
]
]

[
 $p_{\theta}(x_{t-1} \mid x_t)$

$p_{\theta}(x_{t-1} \mid x_t)$
 $\quad \text{= learned approximation}$
 $\}$
 $\]$

And during sampling:

$\[$
 $x_{t-1} \sim p_{\theta}(x_{t-1} \mid x_t).$
 $\]$

The subtle point is that q depends on (x_0) , while p_{θ} does not:

$\[$
 $q(x_{t-1} \mid x_t, \textcolor{blue}{x_0})$
 $\]$

versus

$\[$
 $p_{\theta}(x_{t-1} \mid x_t).$
 $\]$

That's exactly why the neural network is needed: at generation time, (x_0) is unknown, so we need p_{θ} to predict the reverse transition using only (x_t) .

Yes. In DDPM, we intentionally add randomness at every reverse step because $(p_{\theta}(x_{t-1} \mid x_t))$ is a distribution, not just a single deterministic value.

We have

$\[$

$p_{\theta}(x_{t-1} \mid x_t)$

$\mathcal{N}(\mu_{\theta}, \sigma_t^2).$
 $\]$

To sample from this Gaussian:

$\[$
 $\boxed{x_{t-1} = \mu_{\theta} + \sigma_t z}$
 $\]$

where

$$[\\ z \\sim \\mathcal{N}(0, I). \\]$$

Why add (z)?

Suppose we didn't add randomness:

$$[\\ x_{t-1} = \\mu_{\\theta}. \\]$$

Then every reverse step would be deterministic.

Starting from the same (x_T), you would always get exactly the same (x_0).

But the goal of a generative model is to represent a distribution of possible images, not just one fixed image.

For example, the same noisy representation (x_t) can plausibly correspond to slightly different (x_{t-1})'s. The Gaussian

$$[\\ p_{\\theta}(x_{t-1} \\mid x_t) \\]$$

represents that uncertainty.

So:

$$[\\ \\boxed{\\mu_{\\theta} = \\text{most likely/central prediction}} \\]$$

$$[\\ \\boxed{\\sigma_t z = \\text{random variation around that prediction}} \\]$$

Think of it this way

At one step:

[
 x_t
]

↓

network predicts where the previous state should be

[
 μ_θ
]

↓

sample around that prediction

[
 $\mu_\theta + \sigma_t z$
]

↓

[
 x_{t-1}
]

This randomness is what allows different sampling runs to produce different images.

And notice something important: the randomness gets smaller as we approach ($t=0$) because the reverse process becomes increasingly certain.

So the overall process is roughly:

[
 random noise
 $\rightarrow$
 $\text{random/coarse structure}$
 $\rightarrow$
 $\text{more constrained structure}$
 $\rightarrow$
 fine details .
]

In short: we add randomness because we are sampling from the learned probability distribution ($p_\theta(x_{t-1} \mid x_t)$), rather than simply taking its mean. Yes — with one important correction.

Your understanding is basically right:

The forward process is not trained in the original DDPM implementation. The neural network is trained to model the reverse process.

But the connection to a VAE is very important because DDPM can be viewed as a VAE with a very large number of latent variables.

1. First: How is diffusion related to a VAE?

A VAE has:

[
 $x \rightarrow z \rightarrow x$
]

where:

- x = observed data
- z = latent variable
- encoder ($q_{\phi}(z|x)$)
- decoder ($p_{\theta}(x|z)$)

The VAE learns:

[
 $q_{\phi}(z|x)$
]

and

[
 $p_{\theta}(x|z)$
]

Its ELBO is:

[

$\mathbb{E}_{q_{\phi}(z|x)}$

$$[-\log p_{\theta}(x|z)]$$

$$D_{\text{KL}}(q_{\phi}(z|x)||p(z)).$$

]

2. Diffusion does something similar, but with MANY latent variables

Instead of one latent variable (z), DDPM has:

$$[\boxed{x_1, x_2, \dots, x_T}]$$

as latent variables.

The observed data is:

$$[x_0.]$$

So you can think of:

$$[\boxed{x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_T}]$$

as the latent-variable hierarchy.

Here:

$$[x_1, \dots, x_T]$$

are all latent variables.

3. This is why DDPM is called a latent-variable model

The paper explicitly writes:

$$[$$

$$p_{\theta}(x_0)$$

$$\int p_{\theta}(x_{0:T}) dx_{1:T}.$$

This is exactly the same basic idea as a latent-variable model.

You have:

$$[\boxed{x_0 = \text{observed}}]$$

and

$$[\boxed{x_{1:T} = \text{latent}}].$$

The difference is that DDPM has a chain of latent variables, rather than a single latent variable.

4. Compare VAE and DDPM

VAE

[
x
 $\rightarrow$
z
]

and

[
z
 $\rightarrow$
x.
]

Diffusion

[
x₀
 $\rightarrow$
x₁
 $\rightarrow$
x₂
 $\rightarrow$
 $\cdots$
 $\rightarrow$
x_T
]

and reverse:

[
x_T
 $\rightarrow$
x_{T-1}
 $\rightarrow$
 $\cdots$
 $\rightarrow$
x₁
 $\rightarrow$

$x_0.$
]

So you can think of diffusion as a hierarchical/multi-step latent-variable model.

5. The forward process is analogous to the VAE encoder

In a VAE:

[
 $q_{\phi}(z|x)$
]

is the approximate posterior/encoder.

In DDPM:

[
 $\boxed{q(x_{1:T}|x_0)}$
]

is the approximate posterior.

The paper actually calls it:

approximate posterior ($q(x_{1:T}|x_0)$)

and calls it the forward process or diffusion process.

So:

[
 $\boxed{\text{VAE encoder} \quad \quad \quad \text{DDPM forward process}}$
]

6. But there is a huge difference

In a normal VAE:

[
 $q_{\phi}(z|x)$
]

is learned.

The encoder has parameters:

[
 ϕ .
]

For example:

[
 $\mu_{\phi}(x), \sigma_{\phi}(x)$.
]

You train those parameters.

In the original DDPM

The forward process is:

[

$q(x_t|x_{t-1})$

$\mathcal{N}$
 $\left(\sqrt{\alpha_t}x_{t-1}, \beta_t\right)$

\right).
]

The (β_t) 's are fixed beforehand.

Therefore:

[
\boxed{
q\text{ has no trainable parameters.}
}
]

So unlike a VAE encoder:

[
q_\phi(z|x),
]

DDPM uses:

[
\boxed{
q(x_{1:T}|x_0)
}
]

with fixed parameters.

7. So your statement is correct

You said:

"forward process does not train."

For the original DDPM implementation:

[
\boxed{\text{Correct.}}
]

The forward process is a fixed corruption/noising process.

You choose the noise schedule:

```
[
\beta_1,\ldots,\beta_T.
]
```

Then it stays fixed during neural-network training.

8. Then where does the training happen?

The trainable part is the reverse process:

```
[
\boxed{
p_{\theta(x_{t-1}|x_t)}
}
]
```

where:

[

$p_{\theta(x_{t-1}|x_t)}$

```
\mathcal{N}
(
x_{t-1};
\mu_{\theta(x_t,t)},
\sigma_t^2)
]
```

The neural network learns:

```
[
\epsilon_{\theta(x_t,t)}.
]
```

Therefore:

[

$\boxed{}$

$\text{Main trainable component}$

$\text{reverse process}.$
}
]

9. But don't say the forward process is "useless"

This is an important distinction.

The forward process isn't trained, but it is essential for training.

It creates the training examples.

Suppose you have:

[
 $x_0 = \text{real image}.$
]

You randomly choose (t) and (ϵ) .

Then:

[

x_t

$$\sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon.$$

Now you have:

$$(x_t, t, \epsilon).$$

The network sees:

$$(x_t, t)$$

and must predict:

$$\epsilon.$$

So the forward process acts like a data generator for the training task.

10. A very useful analogy with supervised learning

Imagine you want to train a classifier.

You have:

$$\text{image} \rightarrow \text{label}.$$

You don't train the process that creates the dataset.

You simply use it to create training examples.

Similarly, DDPM does:

```
[
x_0
\rightarrow{\text{fixed forward process}}
x_t
]
```

and creates the supervised target:

```
[
\epsilon.
]
```

Then:

```
[
(x_t,t)\rightarrow\epsilon_\theta.
]
```

So:

```
[
\boxed{
\text{forward process creates the training problem}
}
]
```

while:

```
[
\boxed{
\text{reverse network learns the solution}.
}
]
```

11. Now connect this directly to the VAE ELBO

A VAE has:

```
[
\log p_\theta(x)
```


$$\begin{aligned} & \geq \\ & \mathbb{E}_{q_{\phi}(z|x)} \\ & \left[\log p_{\theta}(x,z) - \log q_{\phi}(z|x) \right] \end{aligned}$$

DDPM does essentially the same thing:

$$\begin{aligned} & \left[\log p_{\theta}(x_0) \right. \\ & \geq \\ & \mathbb{E}_q \\ & \left[\log \right. \\ & \left. \frac{p_{\theta}(x_{0:T})}{q(x_{1:T}|x_0)} \right. \\ & \left. \right] \end{aligned}$$

So DDPM is also using variational inference.

The difference is that the approximate posterior is now:

$$\begin{aligned} & \left[q(x_{1:T}|x_0) \right] \end{aligned}$$

instead of:

$$\begin{aligned} & \left[q_{\phi}(z|x) \right] \end{aligned}$$

12. The DDPM ELBO decomposition

The DDPM ELBO becomes:

[

L

$$L_T + L_{T-1} + \cdots + L_1 + L_0.$$

]

Specifically:

[

L_T

$$D_{\text{KL}}(q(x_T|x_0)||p(x_T))$$

]

then:

[

L_{t-1}

$$D_{\text{KL}}(q(x_{t-1}|x_t,x_0) || p_{\theta}(x_{t-1}|x_t))$$

]

and finally:

[

L_0

$-\log p_{\theta}(x_0|x_1).$

]

This is analogous to the VAE's reconstruction and KL terms, but distributed across many timesteps.

13. The most important VAE $\leftrightarrow$ DDPM correspondence

VAE	DDPM
(x)	(x_0)
latent (z)	(x_1,\ldots,x_T)
encoder ($q_{\phi}(z x)$)	
decoder ($p_{\theta}(x z)$)	
prior ($p(z)$)	($p(x_T)=\mathcal{N}(0,I)$)
learned encoder	fixed forward process
learned decoder	learned reverse process

ELBO

diffusion variational
bound

This table is the connection you should keep in mind.

14. Why is (x_T) analogous to the VAE latent (z) ?

In a VAE:

[
 $z \sim p(z)$
]

during generation.

For example:

[
 $z \sim \mathcal{N}(0, I)$.
]

Then:

[
 $z \rightarrow x$.
]

In DDPM:

[
 $x_T \sim \mathcal{N}(0, I)$.
]

Then:

[
 x_T
 $\rightarrow$

$$x_{T-1} \rightarrow \dots \rightarrow x_0.$$

Therefore:

$$[\boxed{x_T \text{ plays a role analogous to the VAE latent } z}]$$

But there's an important difference:

DDPM does not directly decode (x_T) to (x_0) in one step.

It uses:

$$[T]$$

reverse transitions.

15. This explains why diffusion can be viewed as a "very deep VAE"

A rough mental model is:

VAE

$$[z \rightarrow \text{decoder} \rightarrow x]$$

One big generative transformation.

Diffusion

```
[  
x_T  
\rightarrow  
x_{T-1}  
\rightarrow  
x_{T-2}  
\rightarrow  
\cdots  
\rightarrow  
x_0.  
]
```

Many small generative transformations.

So you can think:

```
[  
\boxed{  
\text{Diffusion}  
\approx  
\text{VAE-like latent-variable model with many latent states and many decoder steps}.  
}  
]
```

This isn't literally "a VAE" in every architectural sense, but it is a very useful conceptual connection.

16. One subtle point: the forward process DOES appear in the ELBO

This is where confusion often occurs.

Even though q isn't trained, it appears in:

```
[  
D_{KL}(q|p_{\theta}).  
]
```

Why?

Because we need a reference distribution.

The forward process tells us:

"If I start with a real (x_0), this is what the noisy latent states should look like."

Then we train the reverse model to reproduce that process backward.

So:

```
[  
  \boxed{  
    q=\text{fixed teacher/reference}  
  }  
]
```

and

```
[  
  \boxed{  
    p_{\theta}=\text{learned reverse model}.  
  }  
]
```

17. Think of q as the teacher

This is an excellent mental model.

Forward:

```
[  
  x_0  
  \rightarrow  
  x_1  
  \rightarrow  
  x_2  
  \rightarrow  
  \cdots  
  \rightarrow
```

x_T .
]

This process is completely known.

It tells us what the correct posterior transition should be:

[
 $q(x_{t-1}|x_t, x_0)$.
]

The neural network tries to approximate it without knowing (x_0):

[
 $p_{\theta}(x_{t-1}|x_t)$.
]

So training is essentially:

[
 $\boxed{\text{learn the reverse of a known corruption process}}$.
]
]

18. This also explains why (x_0) disappears during generation

During training:

[
 x_0
 $\rightarrow$
 q
 $\rightarrow$
 x_t
]

and we know (x_0).

Therefore we can calculate:


```
[
q(x_{t-1}|x_t,x_0).
]
```

During generation:

```
[
x_T
\rightarrow
x_{T-1}
\rightarrow
\cdots
]
```

we don't have (x_0) .

Therefore we cannot use:

```
[
q(x_{t-1}|x_t,x_0).
]
```

The neural network has learned an approximation:

```
[
\boxed{
p_{\theta}(x_{t-1}|x_t)
}
]
```

that works using only (x_t) .

19. So what exactly is being "trained"?

This is the cleanest way to say it:

Forward process

```
[
\boxed{
q(x_t|x_{t-1})
}
```

```
}  
]
```

Fixed.

No neural network.

No gradient descent.

No learned parameters in the original DDPM.

Reverse process

```
[  
  \boxed{  
    p_{\theta}(x_{t-1}|x_t)  
  }  
]
```

Learned.

Neural network parameters:

```
[  
  \theta  
]
```

are optimized.

The practical loss is:

```
[  
  \boxed{  
    |\epsilon - \epsilon_{\theta}(x_t, t)|^2.  
  }  
]
```

20. But there is one nuance about "all training happens at reverse"

I'd phrase it slightly differently:

The trainable parameters belong to the reverse process, but the forward process is actively used during every training iteration to construct (x_t) and the target noise.

So don't imagine:

```
[
\text{forward training}
\rightarrow
\text{reverse training}.
]
```

Instead:

```
[
\boxed{
\text{fixed forward process}
\rightarrow
\text{creates training example}
\rightarrow
\text{train reverse network}.
}
]
```

21. One complete training iteration

Suppose you have a real image:

```
[
x_0.
]
```

Step 1: choose timestep

```
[
t=700.
]
```

Step 2: generate Gaussian noise

$$\begin{bmatrix} \epsilon \sim N(0, I). \end{bmatrix}$$

Step 3: use fixed forward process

$$\begin{bmatrix} x_{700} \\ \sqrt{\bar{\alpha}_{700}}x_0 \\ + \\ \sqrt{1-\bar{\alpha}_{700}}\epsilon. \end{bmatrix}$$

Step 4: neural network

$$\begin{bmatrix} \epsilon_{\theta}(x_{700}, 700). \end{bmatrix}$$

Step 5: loss

$$\begin{bmatrix} L = \\ |\epsilon - \epsilon_{\theta}(x_{700}, 700)|^2. \end{bmatrix}$$

Step 6: backpropagation

Update:

$$\begin{bmatrix} \boxed{\theta} \end{bmatrix}$$

only.

The forward-process quantities:

$$\begin{bmatrix} \beta_t, \alpha_t, \bar{\alpha}_t \end{bmatrix}$$

remain fixed.

22. Then generation

After training is complete:

$$\begin{bmatrix} x_T \sim N(0, I). \end{bmatrix}$$

No real (x_0) exists.

Then:

$$\begin{bmatrix} x_T \\ \rightarrow x_{T-1} \\ \rightarrow \cdots \\ \rightarrow x_0. \end{bmatrix}$$

At every step:

$$\begin{bmatrix} \epsilon_\theta(x_t, t) \end{bmatrix}$$

is predicted.

Then:

$$\begin{bmatrix} \mu_\theta(x_t, t) \end{bmatrix}$$

is calculated.

Then:

[

x_{t-1}

$\mu_{\theta} + \sigma_z$
]

23. The entire VAE-to-DDPM picture

I would remember it like this:

```
[
\boxed{
\begin{array}{ccc}
\textbf{VAE} & \& \textbf{DDPM} \\
x \& \leftrightarrow \& x_0 \\
\downarrow q_\phi \& \& \downarrow q; \text{(fixed)} \\
z \& \& x_1, x_2, \dots, x_T \\
\downarrow p_\theta \& \& \downarrow p_\theta \\
x \& \& x_{T-1}, \dots, x_1, x_0 \\
\end{array}
}
]
```

The big conceptual difference is:

```
[
\boxed{
\text{VAE: one learned latent representation}
}
]
```

versus

```
[
\boxed{
```

```
\text{DDPM: a sequence of progressively noisier latent representations}.
}
```

And:

```
[
\boxed{
\text{VAE learns encoder + decoder}
}
```

whereas original DDPM:

```
[
\boxed{
\text{fixes the encoder-like forward process and learns the decoder-like reverse process}.
}
```

So your statement is essentially correct:

The forward diffusion process is fixed, not trained; it supplies the noisy latent states and training targets. The trainable neural network is in the reverse process.

Yes — that is the main intuition, with one refinement.

Why do we add noise in many small steps?

Suppose the clean image is:

```
[
x_0
]
```

and we gradually add noise:

```
[
x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \cdots \rightarrow x_T
]
```

where each step adds only a small amount of Gaussian noise.

For example:

[
 $x_{t-1} \rightarrow x_t$
]

might make the image only slightly noisier.

The reverse network then learns:

[
 $x_t \rightarrow x_{t-1}$
]

i.e. how to remove a small amount of noise.

Why not destroy the image in one giant step?

Imagine doing:

[
 x_0
 $\rightarrow$
 x_T
]

where (x_T) is immediately almost pure Gaussian noise.

Then the reverse network would have to learn:

[
 $\text{pure noise} \rightarrow \text{complete image}$.
]

That's an extremely difficult mapping.

There is essentially no obvious local information telling the model:

"This particular random pattern should become this particular image."

With gradual noising

Instead, we have:


```
[
\text{clean}
\rightarrow
\text{slightly noisy}
\rightarrow
\text{more noisy}
\rightarrow
\cdots
\rightarrow
\text{almost pure noise}.
]
```

The reverse process becomes:

```
[
\text{almost pure noise}
\rightarrow
\text{slightly less noisy}
\rightarrow
\text{less noisy}
\rightarrow
\cdots
\rightarrow
\text{clean}.
]
```

Each individual task is much easier:

```
[
\boxed{
x_t \rightarrow x_{t-1}
}
]
```

rather than trying to directly learn:

```
[
\boxed{
x_T \rightarrow x_0.
}
]
```

But one correction to your wording

It's not exactly that the network cannot distinguish noise from image if we add a large amount at once.

The deeper issue is that large noise destroys information, making the reverse mapping highly ambiguous.

For example, suppose two different images:

[
 $x_0^{\{1\}} = \text{cat}$
]

and

[
 $x_0^{\{2\}} = \text{dog}$
]

are both heavily corrupted.

They could produce very similar noisy observations:

[
 $x_T^{\{1\}} \approx x_T^{\{2\}}.$
]

So from (x_T) alone, recovering the exact original is difficult.

With small noise:

[
 $x_t^{\{1\}}$
]

still looks substantially like a cat, while

[
 $x_t^{\{2\}}$
]

still looks substantially like a dog.

Therefore the reverse model has useful information to work with.

And this is why the Markov chain is useful

Instead of learning:

```
[  
\boxed{\text{noise}\rightarrow\text{image}}  
]
```

in one giant jump, diffusion learns:

```
[  
\boxed{  
\text{noise}  
\rightarrow  
\text{slightly cleaner}  
\rightarrow  
\text{slightly cleaner}  
\rightarrow  
\cdots  
\rightarrow  
\text{image}  
}  
]
```

Each reverse transition is a small denoising problem.

That's the central intuition behind diffusion models.